Natural Language Processing Concepts and RNNs

Certificate Course on Artificial Intelligence & Deep Learning
by IIT Roorkee

Index

- **Explore RNNs (Stateless and Stateful)**
  - **character RNN** – Predict the next character in sentence
- **Perform sentiment analysis with RNN (movie reviews)**
- **Encoder–Decoder architecture**
  - Performing neural machine translation (NMT)
- **Attention mechanisms**
  - **RNN-Based**
  - **Transformer**
- **Recent Advances – GPT-2 and BERT**

Natural Language Processing

Let's start with a simple and fun model

that can write like **Shakespeare**

Natural Language Processing

Let's start with a simple and fun model

that can write like **Shakespeare**

A common approach for natural language tasks is to use recurrent neural networks.

CLOUD x LAB

What is a char-RNN?

Char-RNN

- Char-RNN [“character recurrent neural network”],
  - Predicts the next character in a sentence

				P	r	e	d	i	c
					N	o	r	m	a
H	o	w		a	r	e		y	o

t
l
u

Char-RNN

- We will first use a **stateless RNN**
  - That learns on random portions of text per iteration
  - Without information on rest of the text
  - To generate fake Shakespearean text

Generating Shakespearean Text Using a Char-RNN

Generating Shakespearean Text Using a Char-RNN

- In a famous 2015 blog post **Andrej Karpathy** showed how to train an RNN to predict the next character in a sentence

Generating Shakespearean Text Using a Char-RNN

- This Char-RNN can then be used to generate novel text one character at a time

Generating Shakespearean Text Using a Char-RNN

- Here is a small sample of the text generated:

PANDARUS:
Alas, I think he shall be come approached and the day
When little srain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

CLOUD x LAB

Generating Shakespearean Text Using a Char-RNN

Let's look at how to build this Char-RNN, step by step!

Generating Shakespearean Text Using a Char-RNN

- Before we start
  - Loading
  - Preparing
  - Splitting the dataset
- Here are a few points to remember about splitting a sequential dataset

Generating Shakespearean Text Using a Char-RNN

- No shuffling of individual data
- No overlap between training set, validation set, and test set
- Preferably leave a gap between these sets

CLOUD x LAB

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_i.ipynb

CLOUD x LAB

Generating Shakespearean Text Using a Char-RNN

- This model cannot learn patterns longer than `n_steps`
  - Which is just 100 characters
- Even LSTM and GRU cells cannot handle very long sequences

Generating Shakespearean Text Using a Char-RNN

What is the solution to solve this problem?

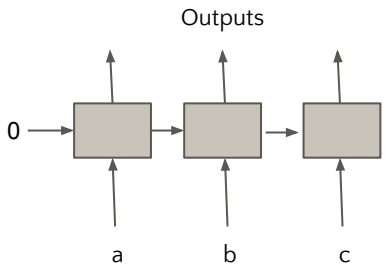
We will use a **stateful RNN**.

Stateful RNN

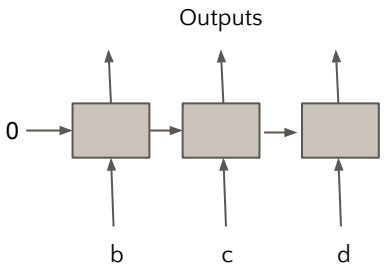
Stateful vs **Stateless** RNN

Stateless RNN:
On every sequence, the input from previous is 0.

Input: abcd
Sequences: [abc, bcd ...]



First Sequence

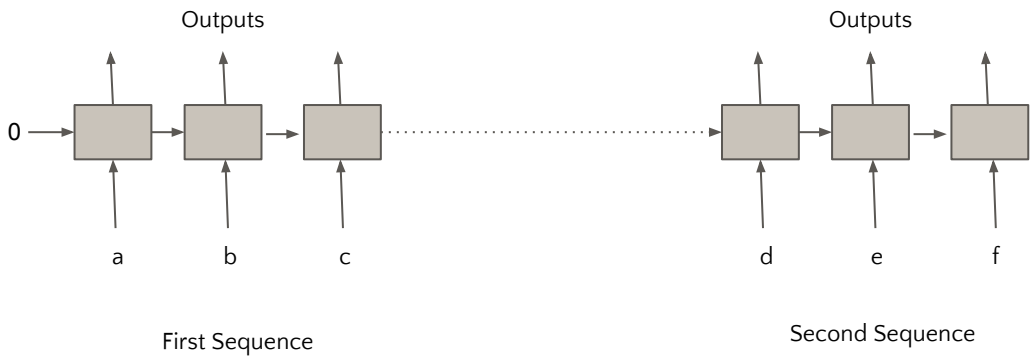


Second Sequence

Stateful vs Stateless RNN

Stateful RNN:
On every (non-overlapping) sequence, the output from previous is fed.

Input: abcdef
Sequences: [abc, def]



CLOUD x LAB

Stateful RNN

- **Stateful RNN**
  - Preserve final state after processing one training batch
  - Then use it as initial state for next training batch

Stateful RNN

Advantage of using a stateful RNN

- Learn long-term patterns despite only backpropagating through short sequences

CLOUD x LAB

Generating Shakespearean Text Using a Char-RNN

Let's learn how to implement a stateful RNN

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_i.ipynb

CLOUD x LAB

Sentiment Analysis

The IMDb Dataset

- IMDb reviews dataset is the “hello world” of natural language processing

The IMDb Dataset

- The dataset from the Internet Movie Database consists of
  - 50,000 movie reviews in English
    - 25,000 for training
    - 25,000 for testing
  - Binary target for each review indicating negative (0) or positive (1)

The IMDb Dataset

How to create a sentiment analysis model around this dataset using NLP?

Let's find out!

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_i.ipynb

CLOUD x LAB

Preprocessing the Original IMDb Dataset

- In a real project, you will preprocess the text yourself
- So, here we will preprocess using only TensorFlow operations

CLOUD x LAB

Switch to Notebook

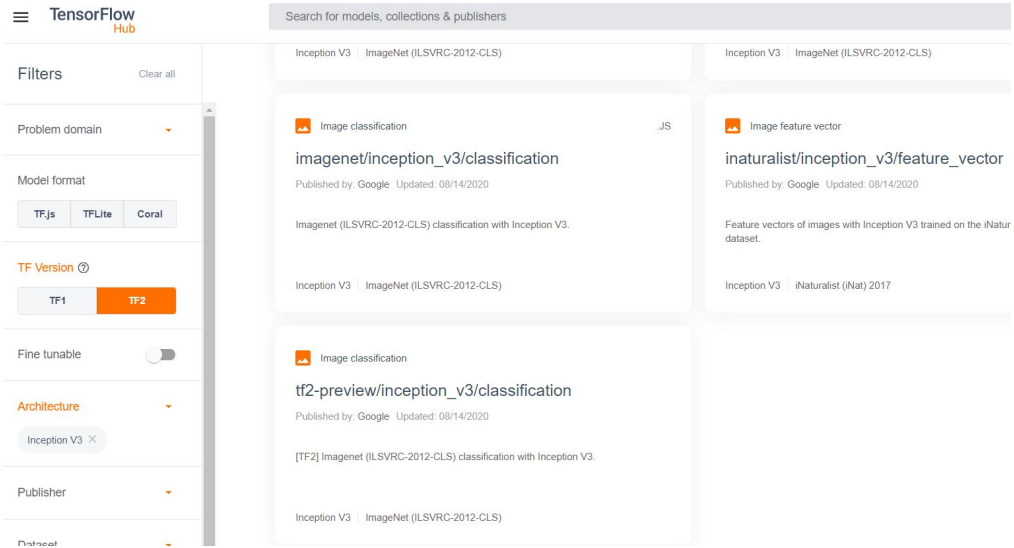
nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_i.ipynb

CLOUD x LAB

Reusing Pretrained Embeddings

Reusing Pretrained Embeddings

- TensorFlow Hub project makes it easy to reuse pretrained model components called `modules`



Reusing Pretrained Embeddings

How to use the modules in your project?

Simple!

Reusing Pretrained Embeddings

- Browse [TF Hub repository]
- Find the module
- Copy the code example

The module will be automatically downloaded, along with its pretrained weights, and included in your model

Easy!

Reusing Pretrained Embeddings

Let's use the `nnlm-en-dim50` sentence embedding module, version 1, in our sentiment analysis model.

CLOUD x LAB

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_ii.ipynb

CLOUD x LAB

Reusing Pretrained Embeddings

Point to remember while using these modules:

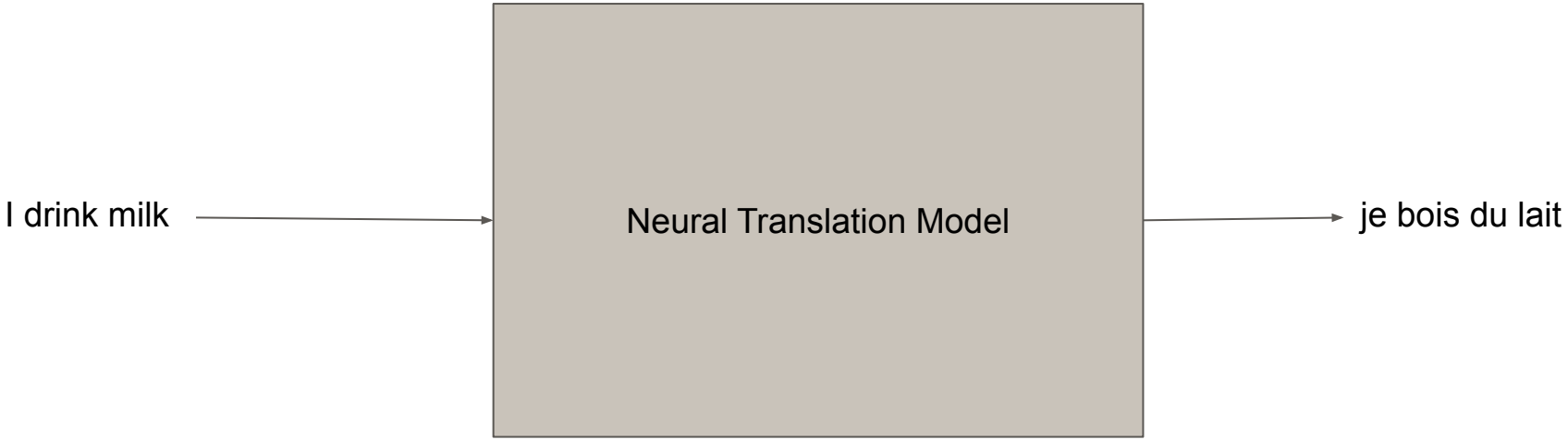
- Use a module that supports TensorFlow 2
- Use the correct version for that module

CLOUD x LAB

An Encoder–Decoder Network for Neural Machine Translation

Neural Machine Translation

A simple Machine Translation Model that translates English sentences to French

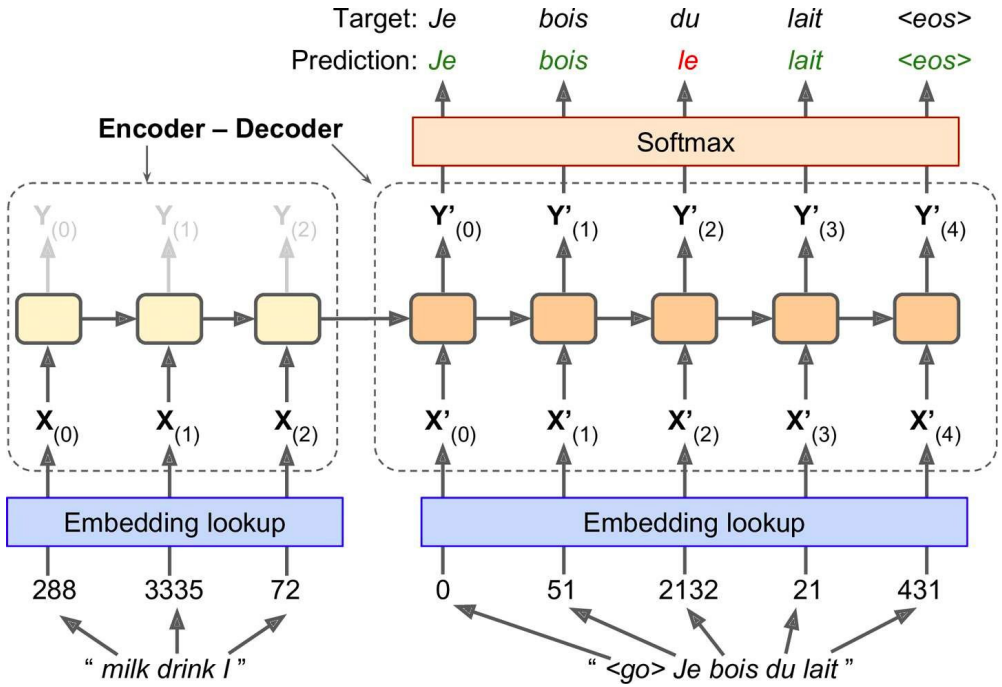


How will you build it?

CLOUD x LAB

Encoder–Decoder Network for Neural Machine Translation

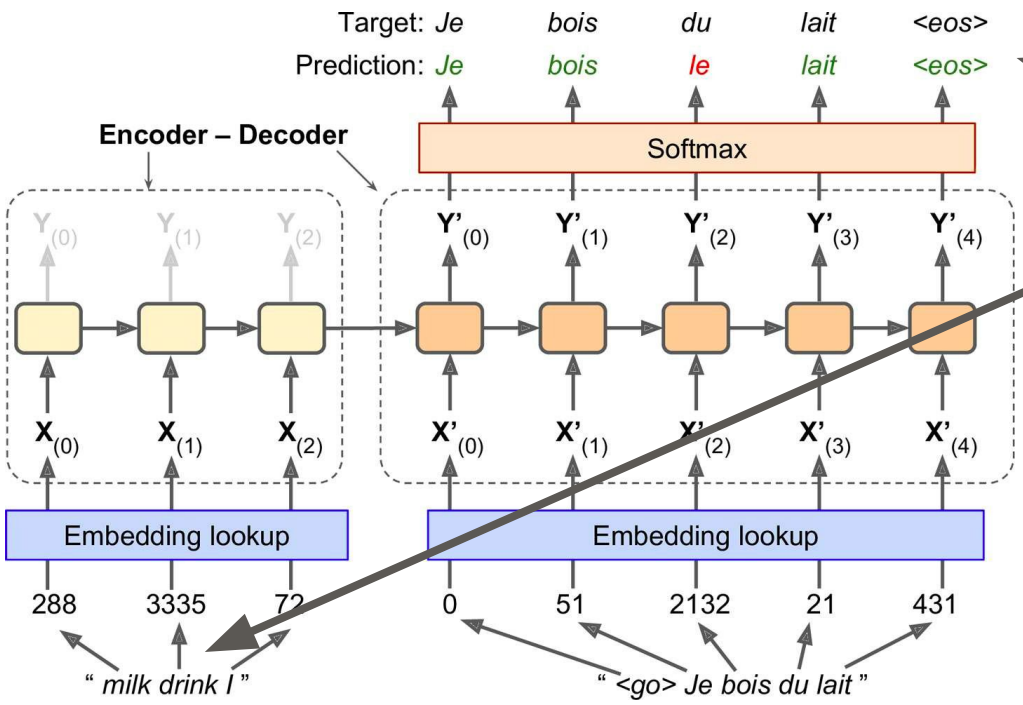
A simple Machine Translation Model that translates English sentences to French



Encoder–Decoder Network for Neural Machine Translation

Let's learn how this Encoder–Decoder Network for Machine Translation is **trained**

Encoder–Decoder Network for Neural Machine Translation

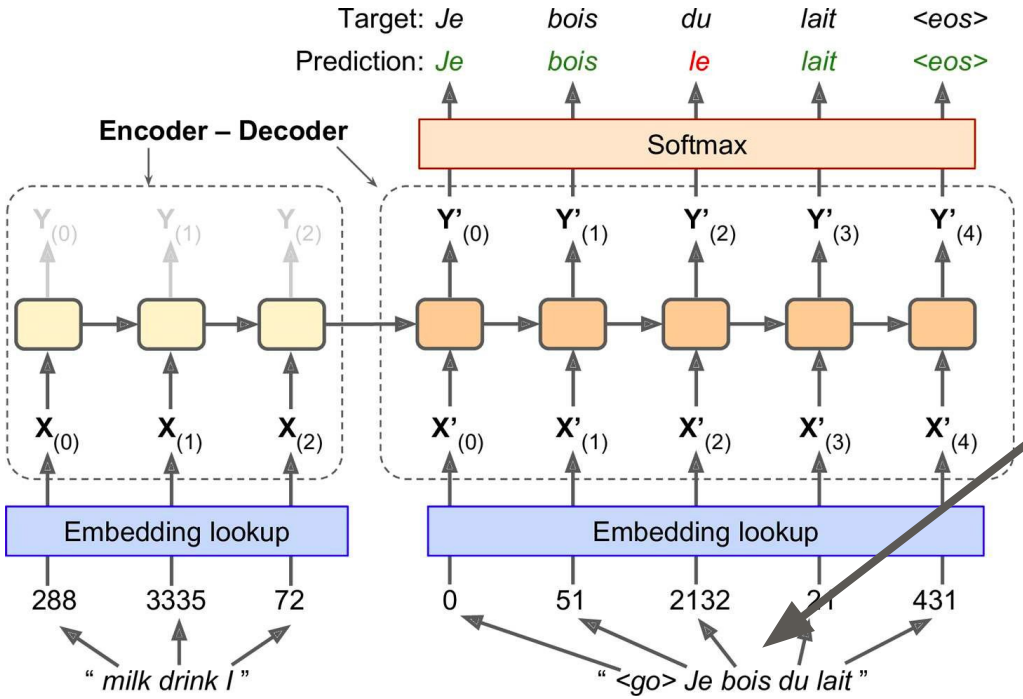


The English sentences are fed to the **encoder**

The **decoder** outputs the French translations

CLOUD x LAB

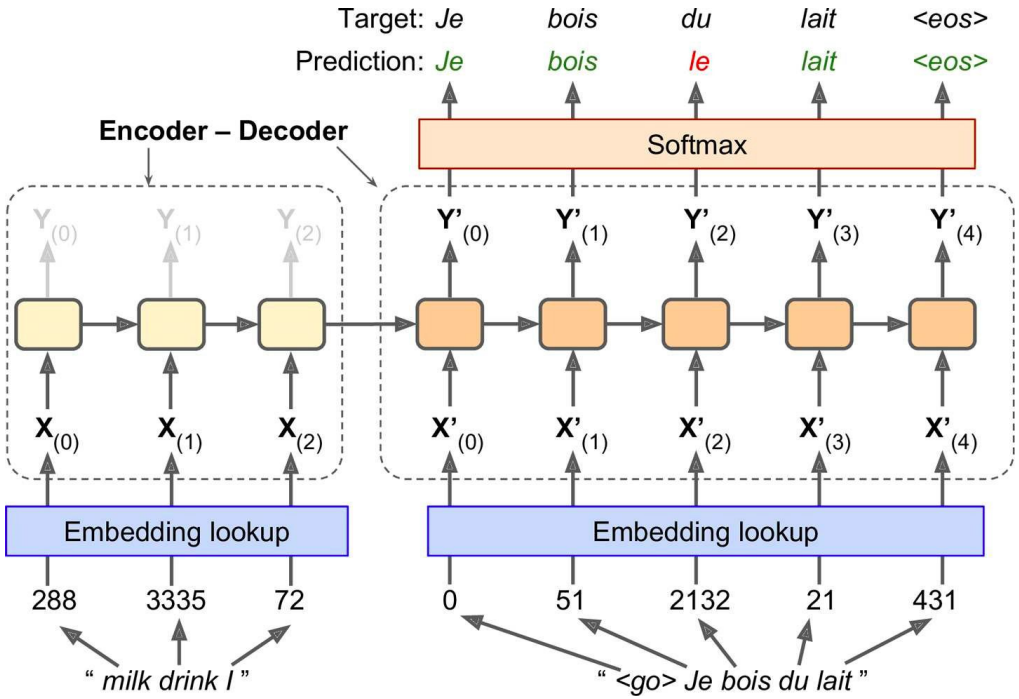
Encoder–Decoder Network for Neural Machine Translation



French translations are also used as inputs to the decoder pushed back by one step

CLOUD x LAB

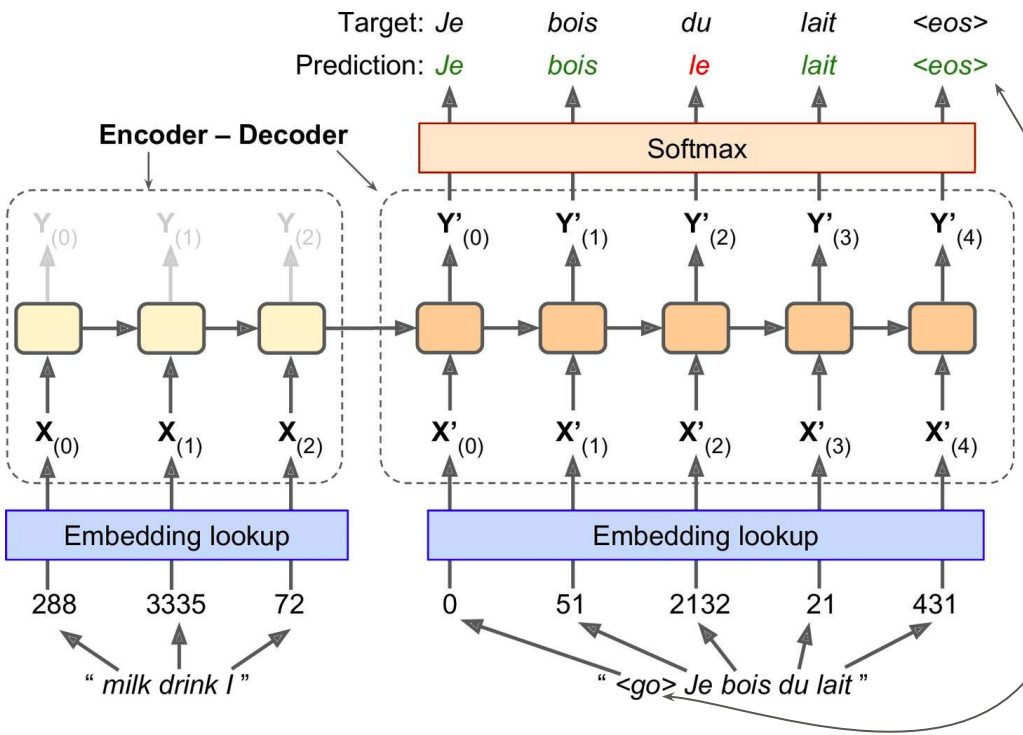
Encoder–Decoder Network for Neural Machine Translation



Decoder is given as input the word that **it should** have output **at the previous step** regardless of what it actually output at the current step.

CLOUD x LAB

Encoder–Decoder Network for Neural Machine Translation

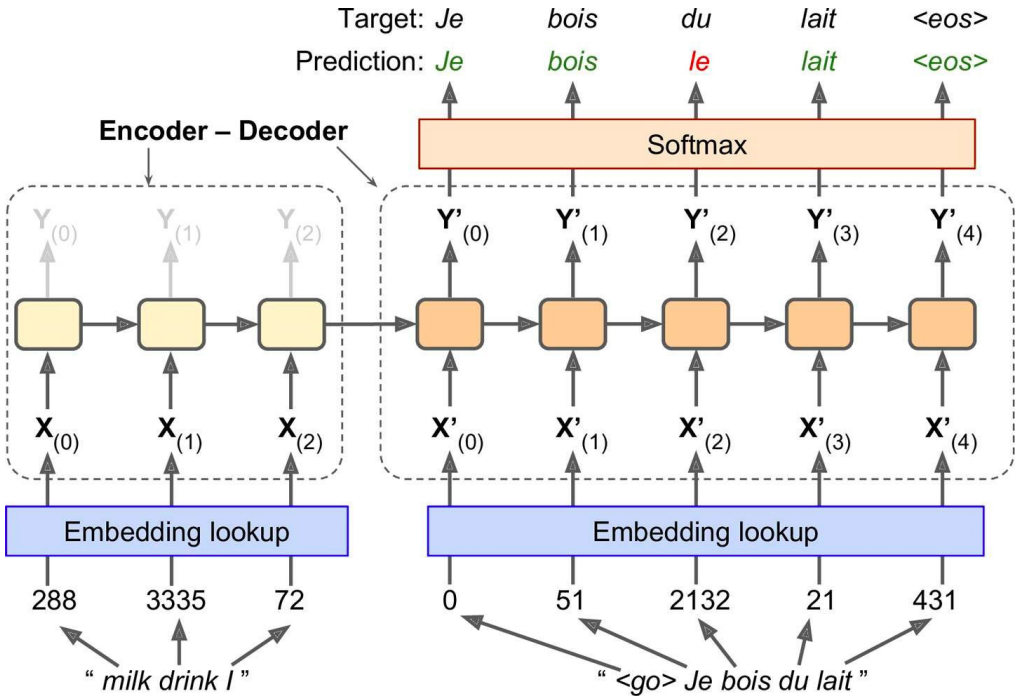


For the first word, **the decoder** is given token that represents the beginning of sentence (here, **"<go>"**)

The decoder is expected to end sentence with end-of-sequence (EOS) token (here, **"<eos>"**)

CLOUD x LAB

Encoder–Decoder Network for Neural Machine Translation

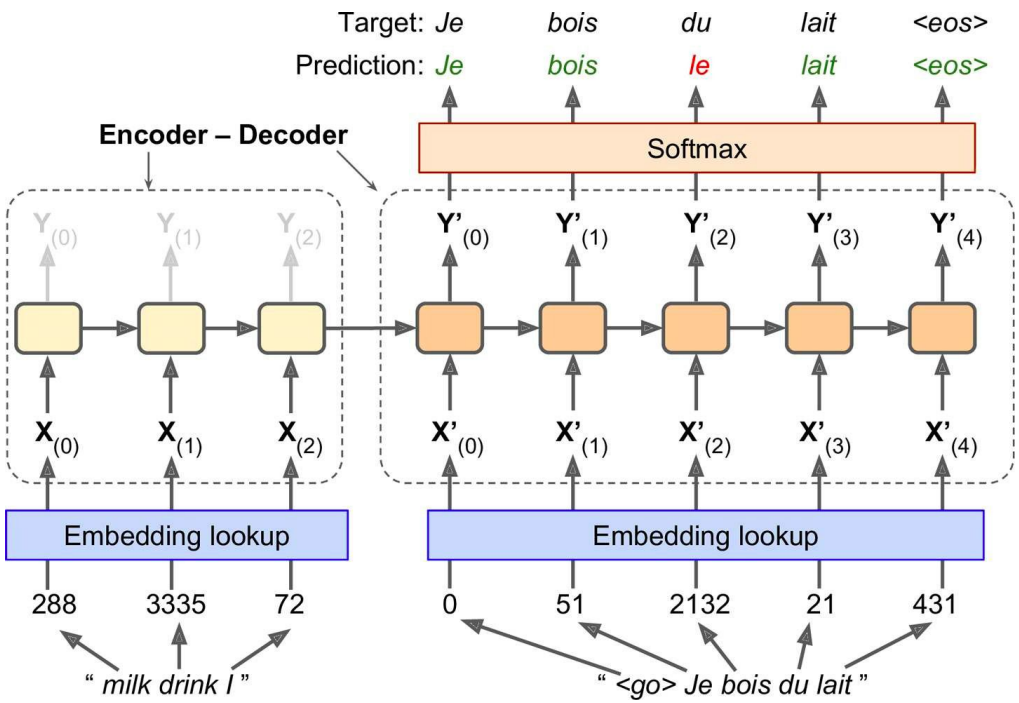


Question:
Why are the English sentences reversed before feeding it to the encoder??

Here **"I drink milk"** is reversed to **"milk drink I"**

CLOUD x LAB

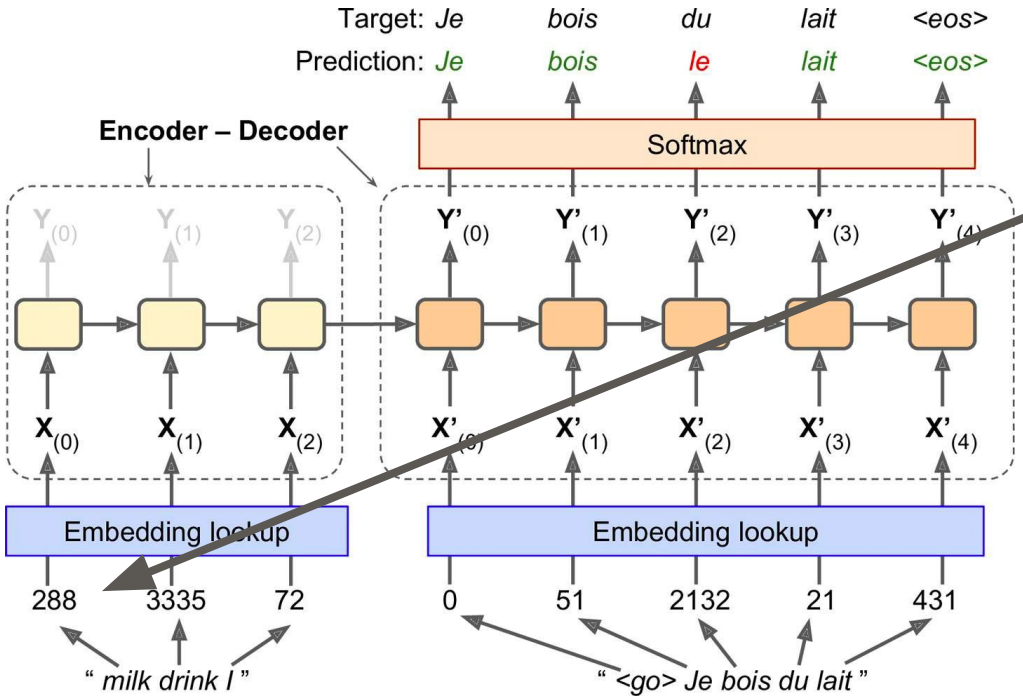
Encoder–Decoder Network for Neural Machine Translation



Answer:
So that the beginning of the English sentence will be fed last to the encoder.

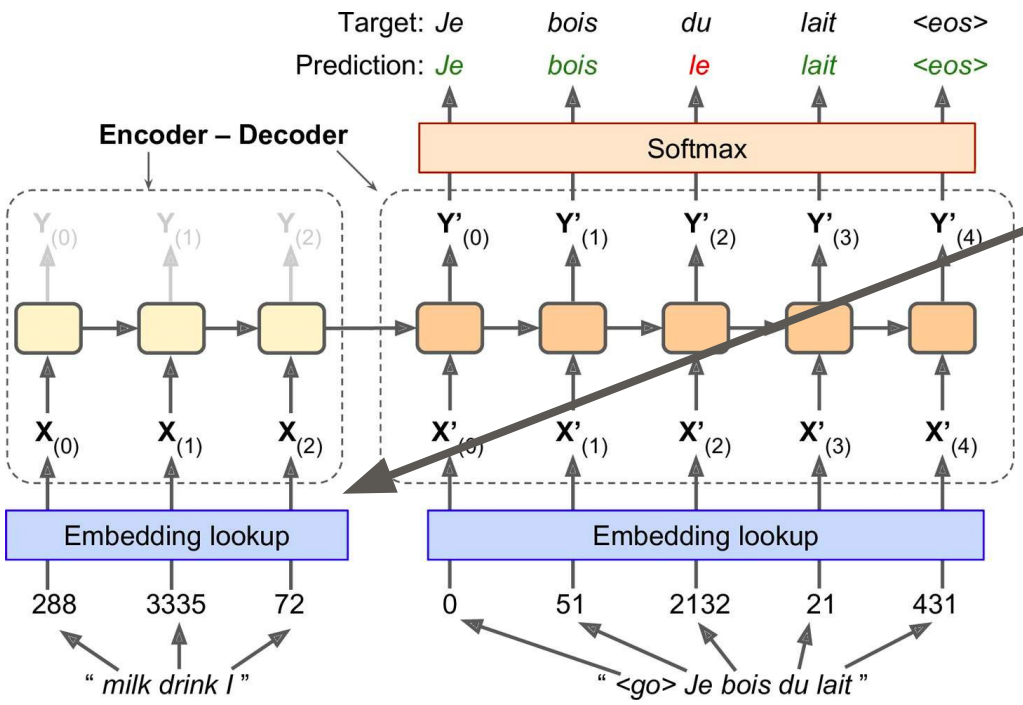
That's generally the first thing that the decoder needs to translate.

Encoder–Decoder Network for Neural Machine Translation



- Each word is initially represented by a simple integer identifier
- e.g., 288 for the word "milk"

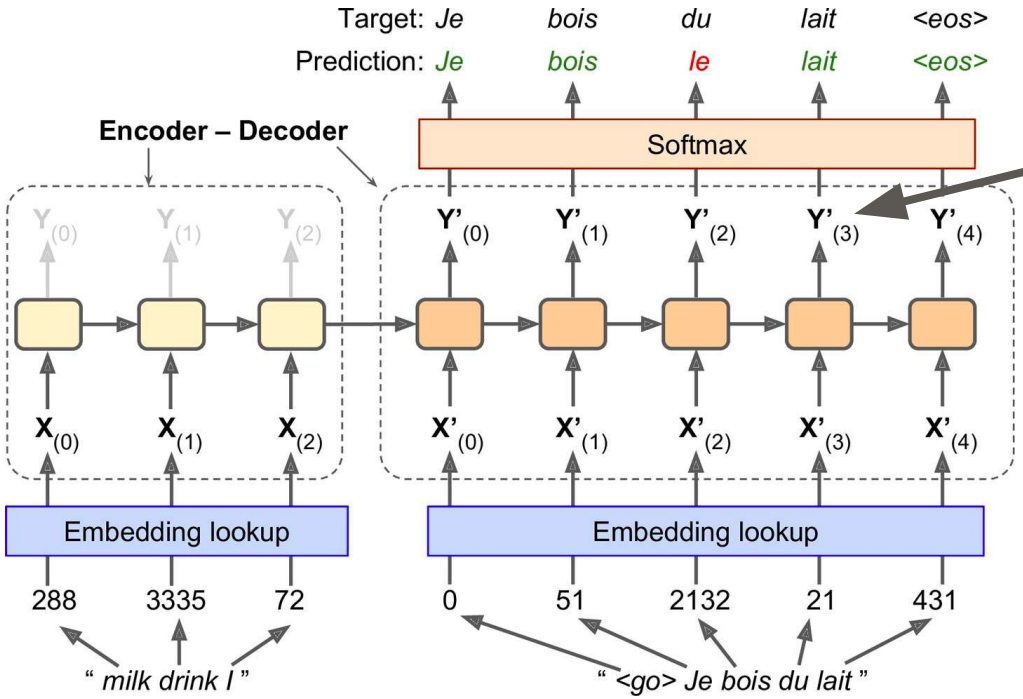
Encoder–Decoder Network for Neural Machine Translation



- Next, an embedding lookup returns the word embedding
- This is a dense, fairly low-dimensional vector
- These word embeddings are what is actually fed to the encoder and the decoder

CLOUD x LAB

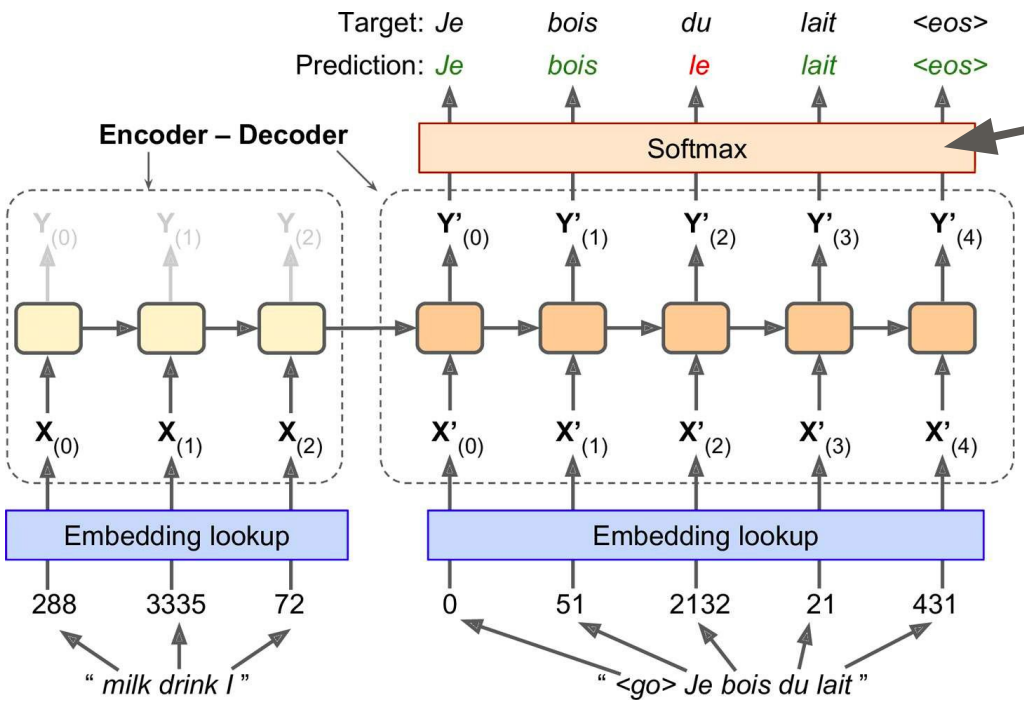
Encoder–Decoder Network for Neural Machine Translation



- At each step, the decoder outputs a score for each word in the output vocabulary i.e., French

CLOUD x LAB

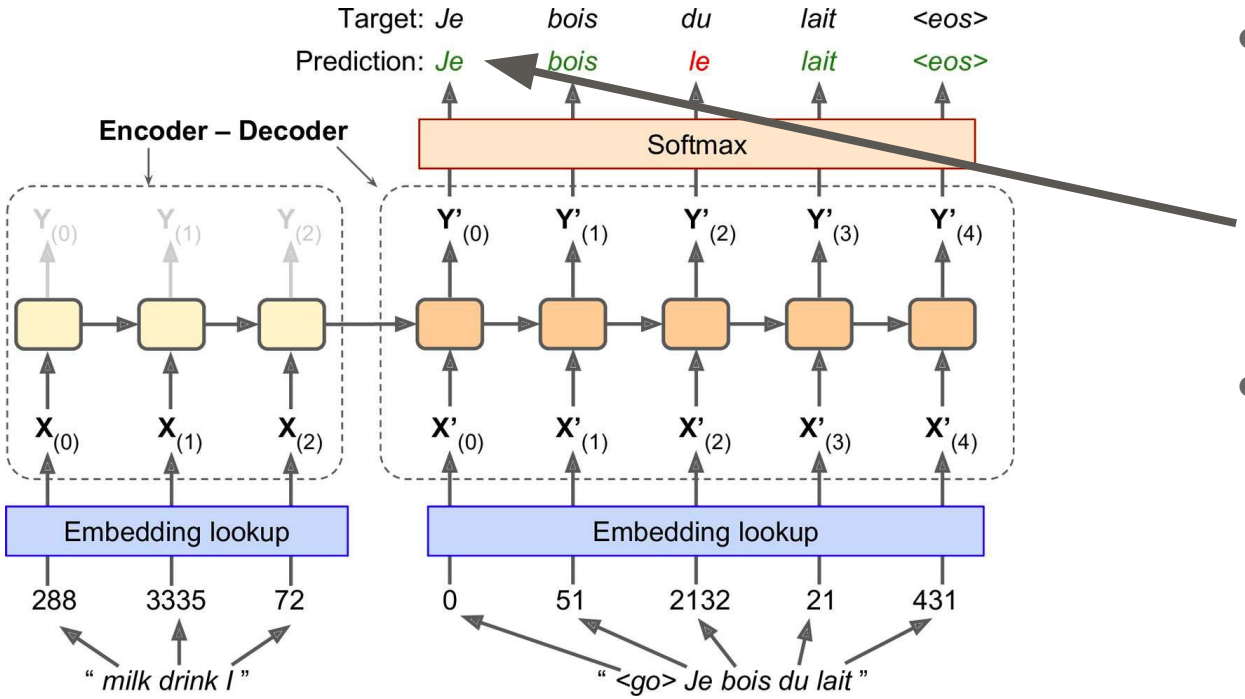
Encoder–Decoder Network for Neural Machine Translation



- Then **Softmax layer** turns these scores into probabilities

CLOUD x LAB

Encoder–Decoder Network for Neural Machine Translation



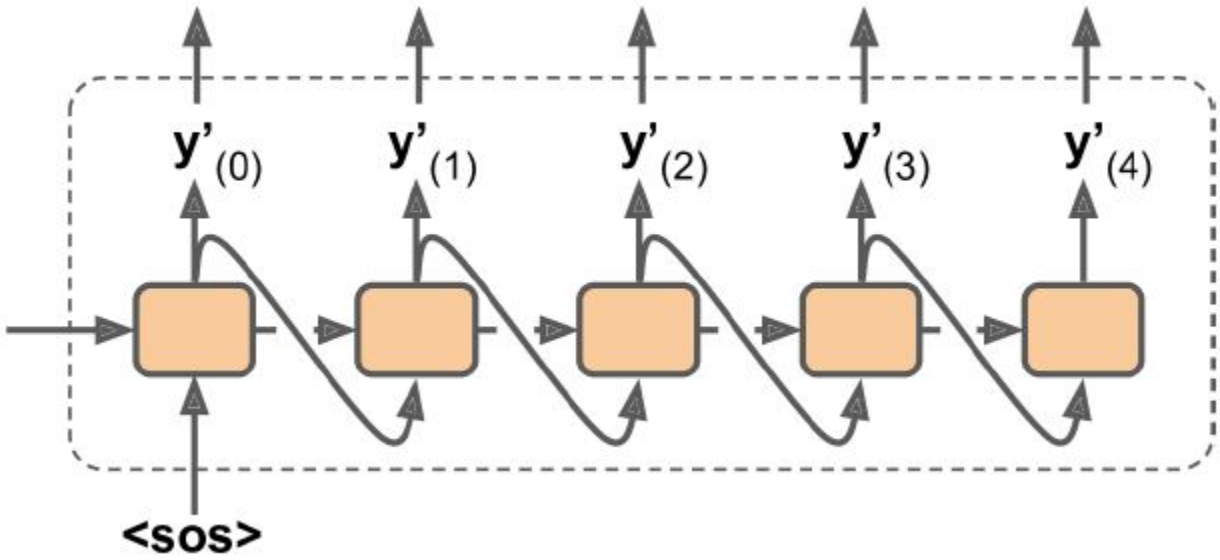
- For example, at the first step "Je" may have a probability of 20%, "Tu" may have a probability of 1%, and so on
- The word with the highest probability is output

Encoder–Decoder Network for Neural Machine Translation

How can we use this Encoder–Decoder Network for Machine Translation at the inference time, since we will not have the target sentence to feed to the decoder?

Encoder–Decoder Network for Neural Machine Translation

- Feed the decoder the word that it output at the previous step
- This requires an embedding lookup that is not shown on the diagram



CLOUD x LAB

Encoder–Decoder Network for Neural Machine Translation

- The TensorFlow Addons project includes many sequence-to-sequence tools to let you easily build production-ready Encoder–Decoders

Encoder–Decoder Network for Neural Machine Translation

Let's see an example.

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_ii.ipynb

CLOUD x LAB

Bidirectional RNNs

Bidirectional RNNs

- NLP tasks, such as Neural Machine Translation, it is preferable to
  - Look ahead at the next words
  - Before encoding a given word

CLOUD x LAB

Bidirectional RNNs

For example:

- To properly encode the word “queen” in
- “the Queen of the United Kingdom,” “the queen of hearts,” and “the queen bee”
- You need to look ahead

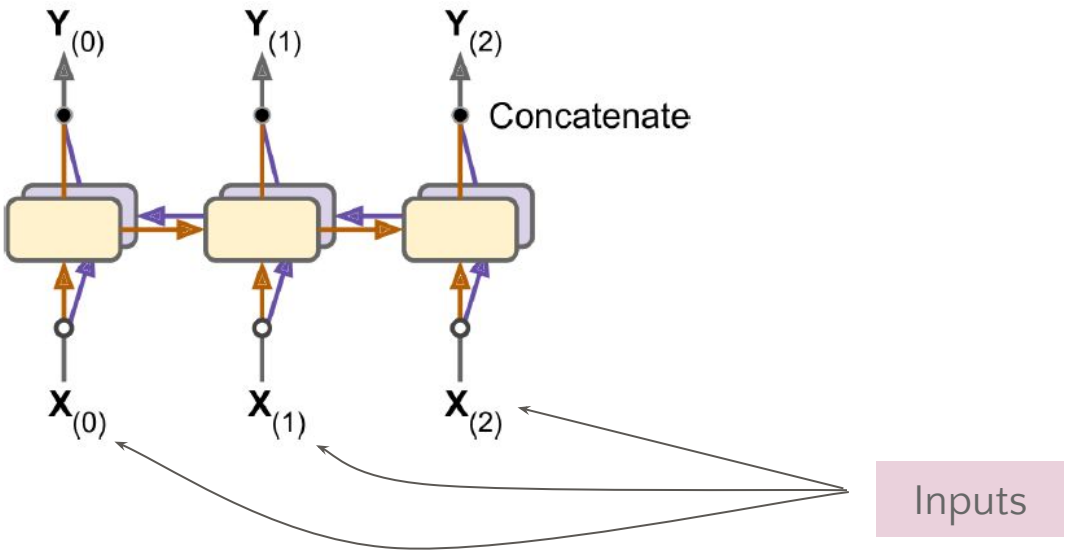
CLOUD x LAB

Bidirectional RNNs

This can be implemented using a **Bidirectional RNN**

Bidirectional RNNs – Implementation

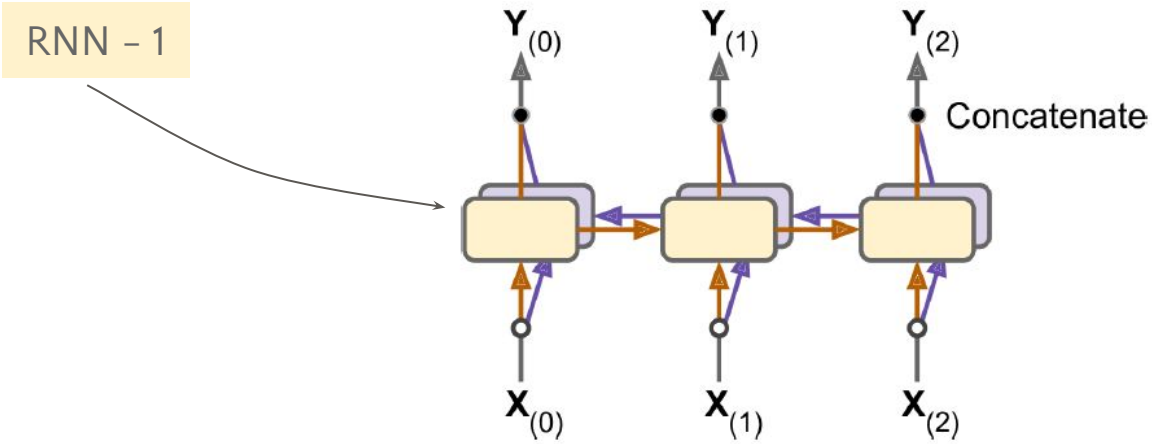
We run two recurrent layers on the same inputs



CLOUD x LAB

Bidirectional RNNs – Implementation

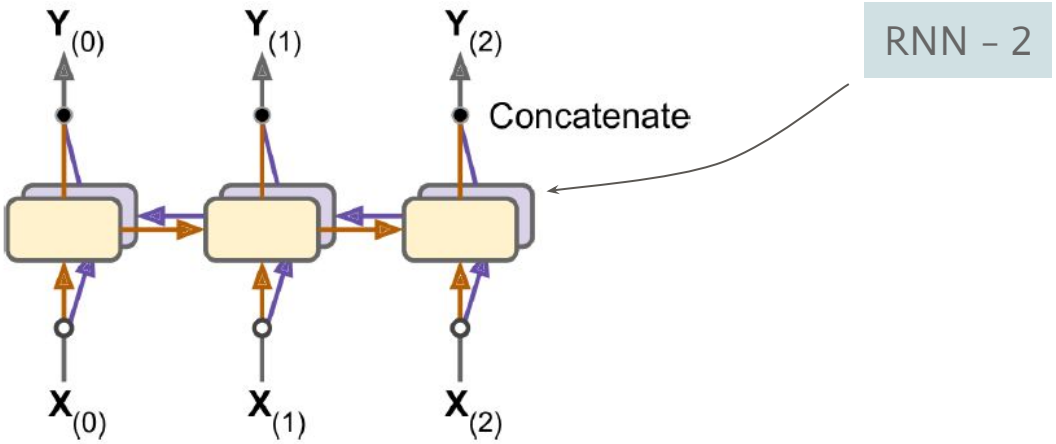
RNN –1 reads the words from left to right



CLOUD x LAB

Bidirectional RNNs – Implementation

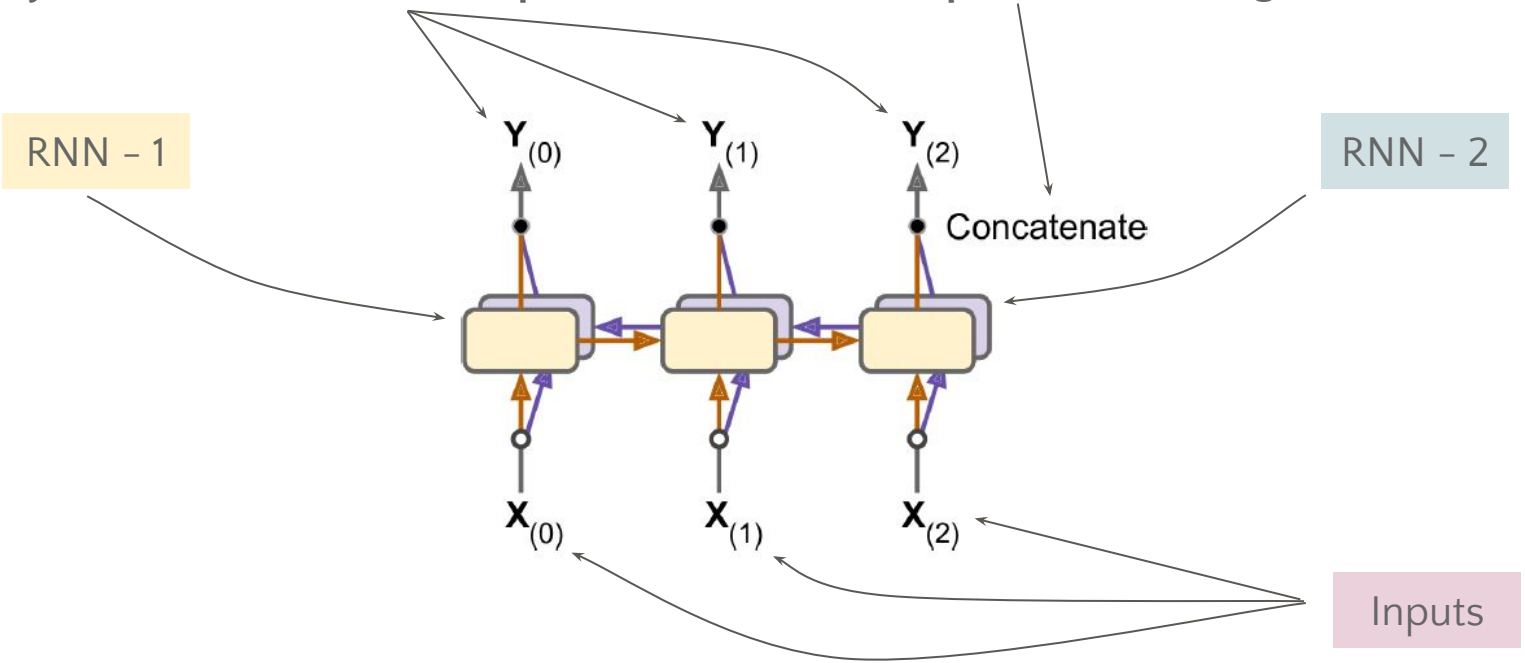
RNN -1 reads the words from right to left



CLOUD x LAB

Bidirectional RNNs – Implementation

Finally, we combine their outputs at each time step concatenating them



CLOUD x LAB

Bidirectional RNNs

How to implement a **Bidirectional RNN** in Keras?

Switch to Notebook

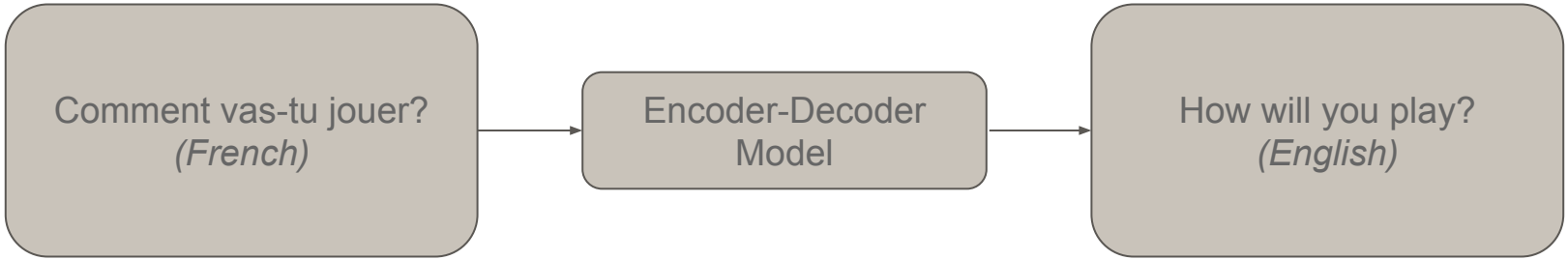
nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_ii.ipynb

CLOUD x LAB

Beam Search

Beam Search

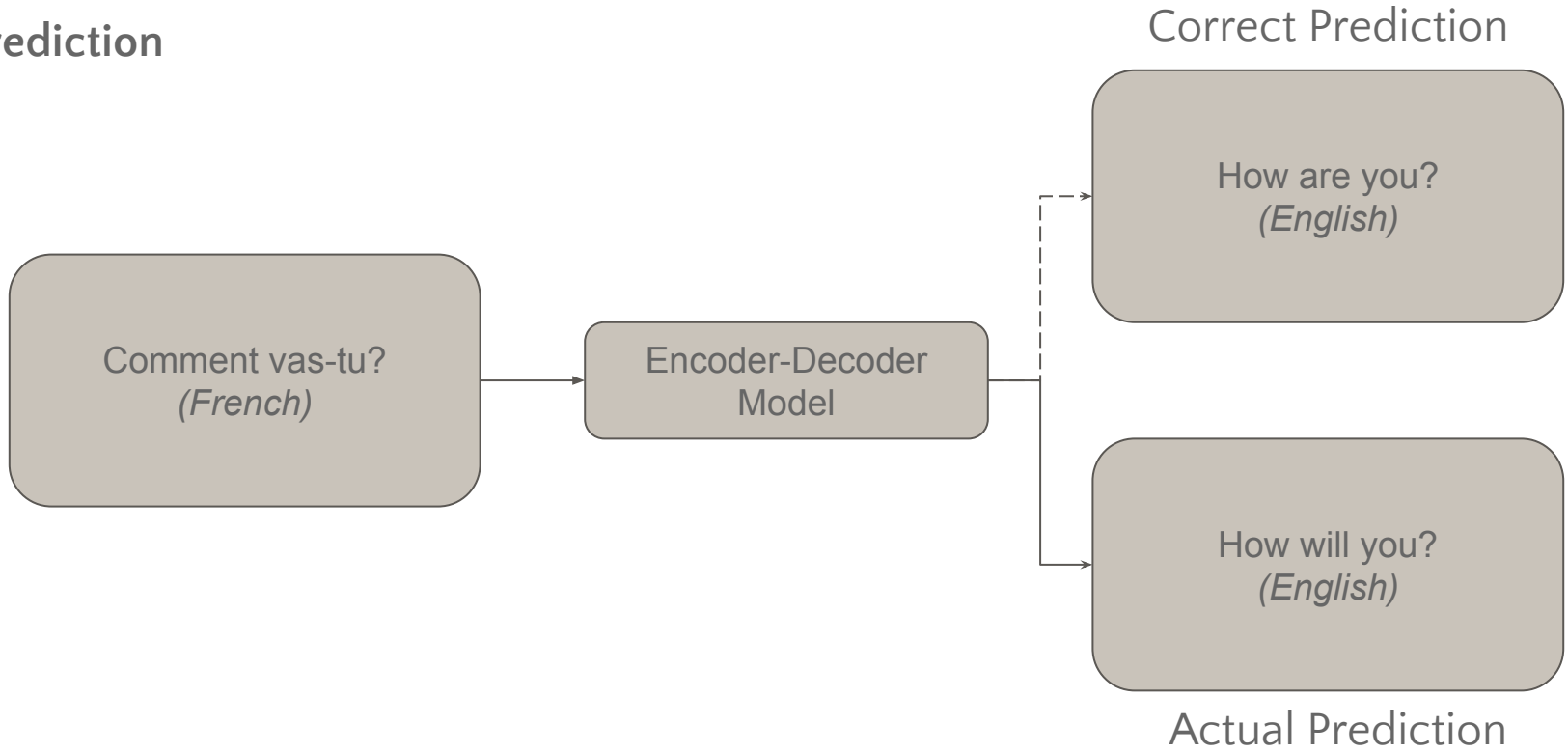
Training



CLOUD x LAB

Beam Search

Prediction



CLOUD x LAB

Beam Search

We fix this using **beam search**

- It keeps track of k (**beam width**) most promising sentences
- At each decoder step
  - It tries to extend them by one word
  - Keeping only the k most likely sentences

Beam Search (width – 3)

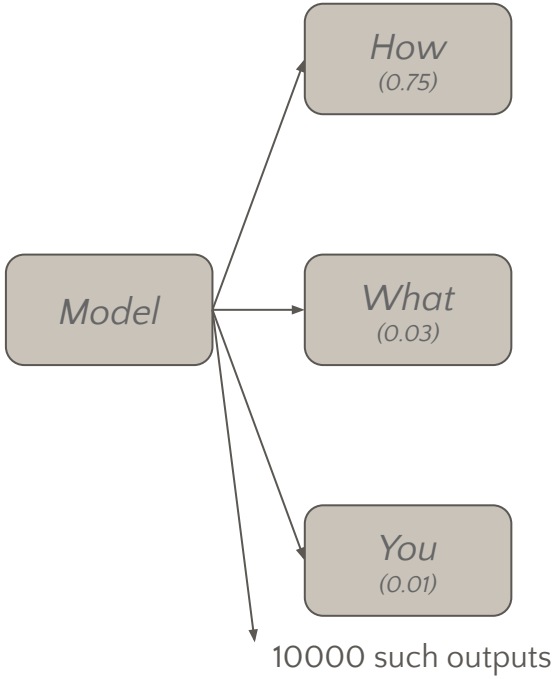
Translating "Comment vas-tu?"

Model

CLOUD x LAB

Beam Search (width – 3)

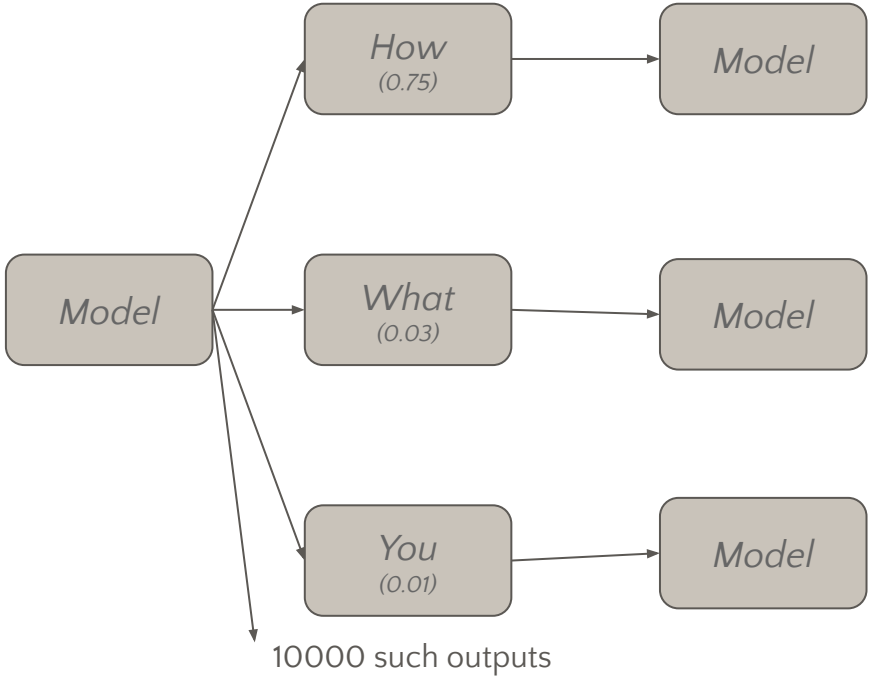
Translating "Comment vas-tu?"



CLOUD x LAB

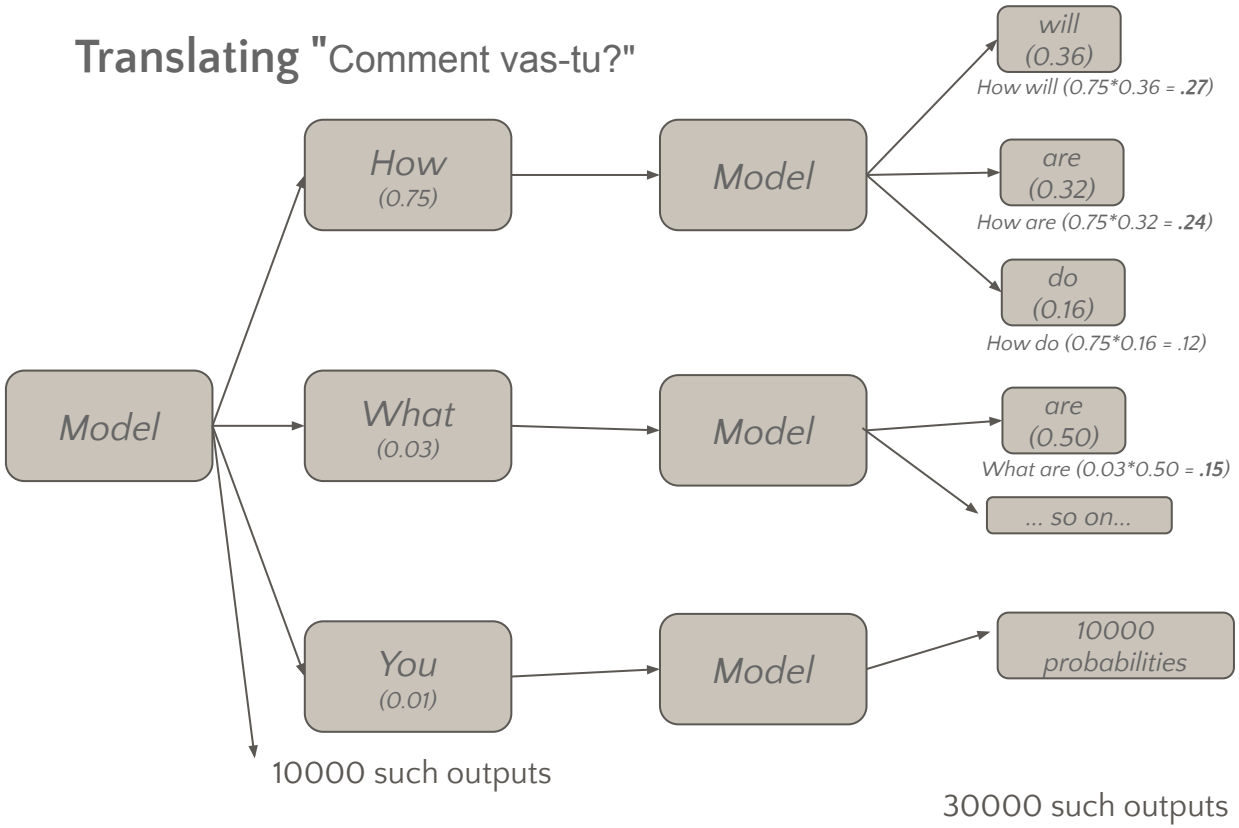
Beam Search (width – 3)

Translating "Comment vas-tu?"



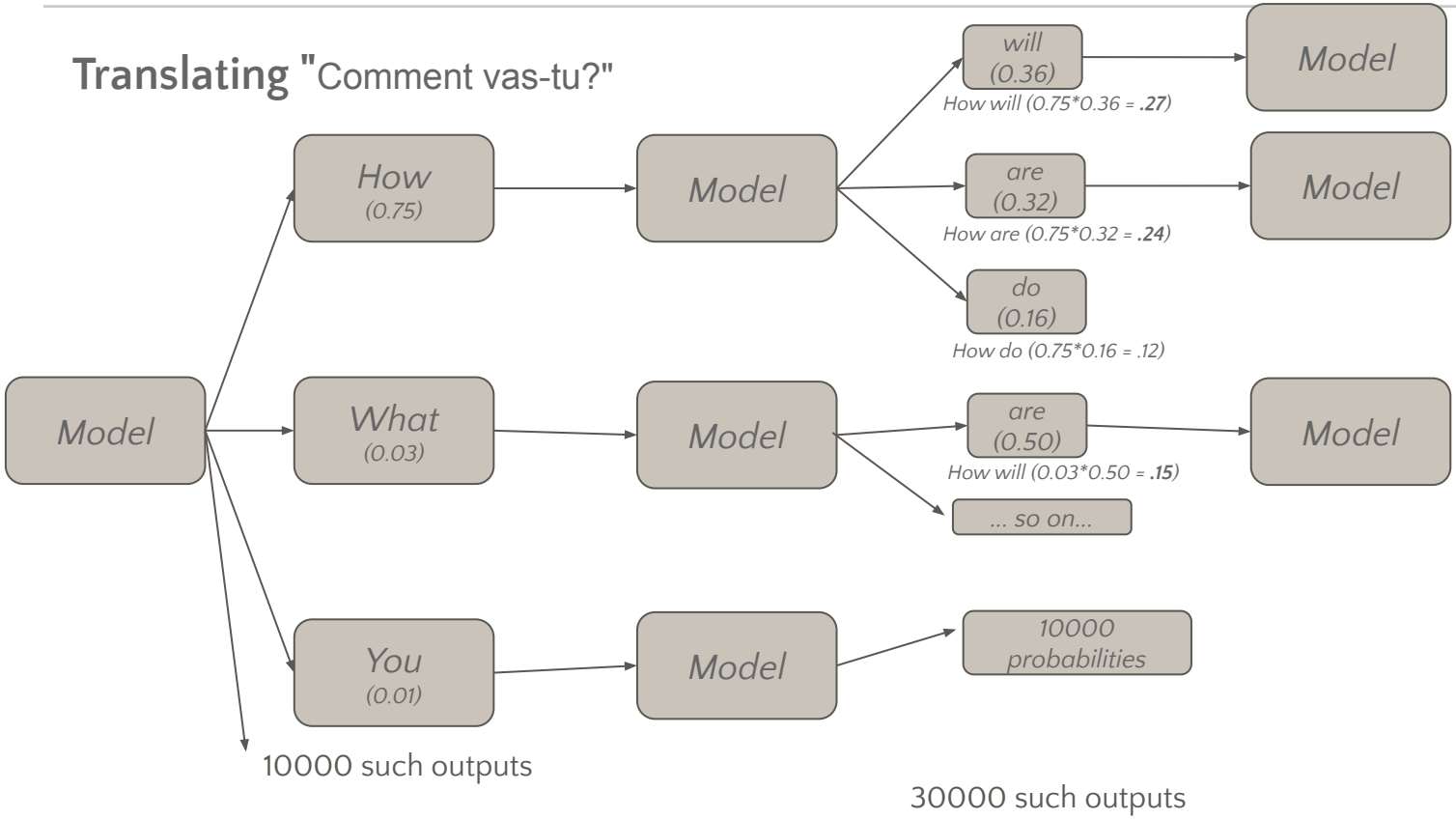
CLOUD x LAB

Beam Search (width – 3)



CLOUD x LAB

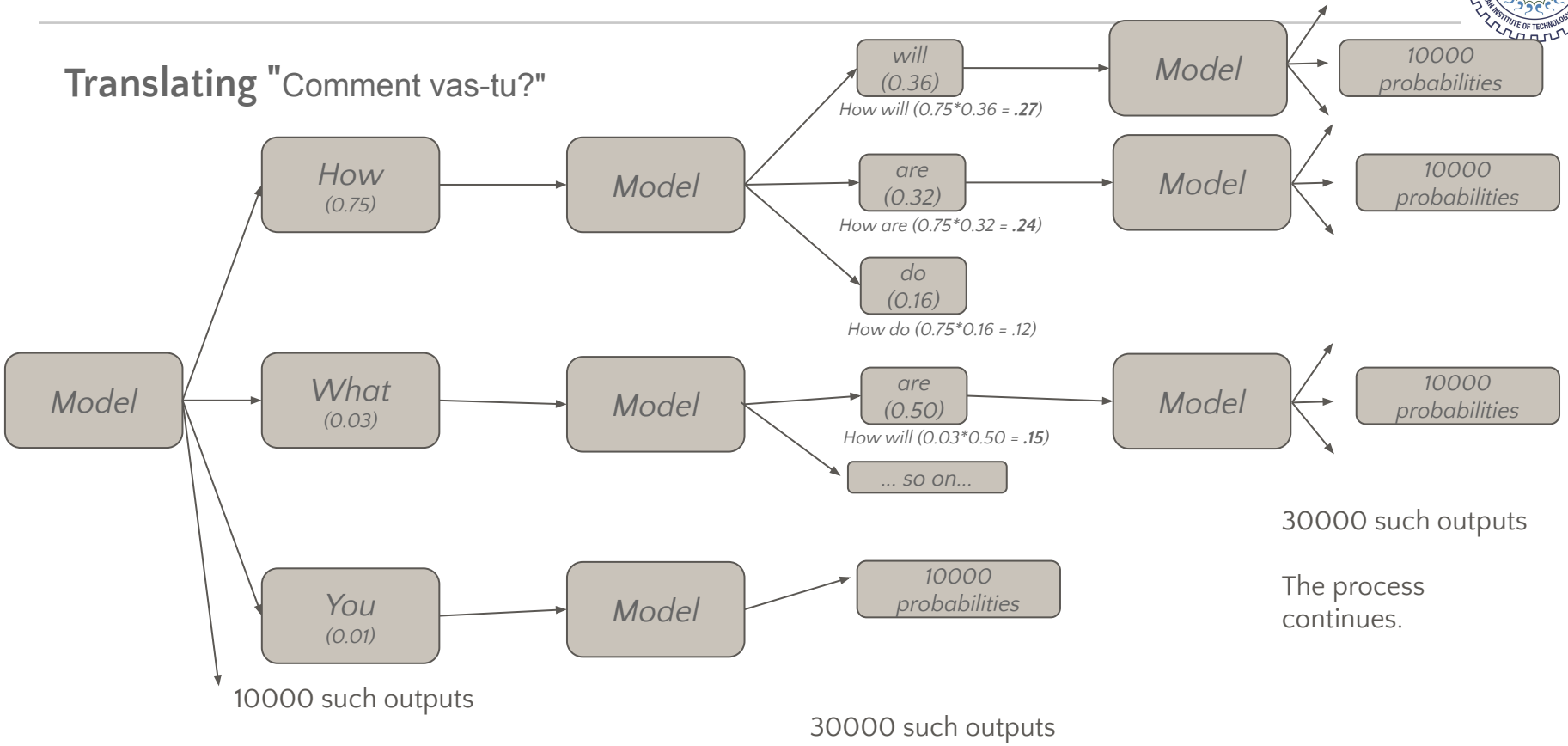
Beam Search (width – 3)



CLOUD x LAB

Beam Search (width – 3)

Translating "Comment vas-tu?"



CLOUD x LAB

Beam Search

- You can implement beam search fairly easily using TensorFlow Addons

```
beam_width = 10
decoder = tfa.seq2seq.beam_search_decoder.BeamSearchDecoder(
    cell=decoder_cell, beam_width=beam_width, output_layer=output_layer)
decoder_initial_state = tfa.seq2seq.beam_search_decoder.tile_batch(
    encoder_state, multiplier=beam_width)
outputs, _, _ = decoder(
    embedding_decoder, start_tokens=start_tokens, end_token=end_token,
    initial_state=decoder_initial_state)
```

Note: This code is given just for explanation and is not implemented in the notebook.

Beam Search

- Unfortunately, this model will be bad because of limited short term memory
- We address this problem using **attention mechanisms**

CLOUD x LAB

Attention Mechanisms

Attention Mechanisms

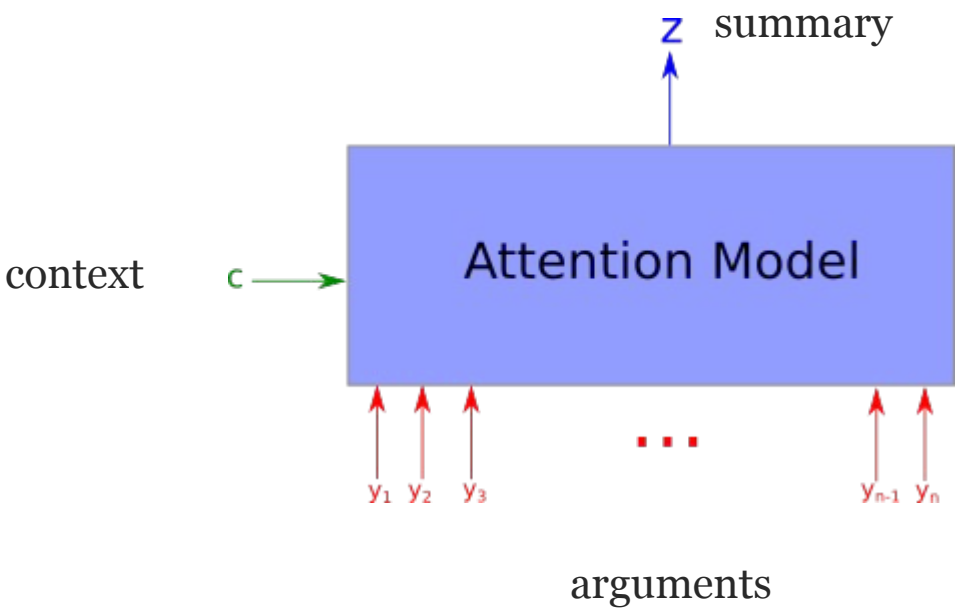
- In a 2014 paper, Dzmitry Bahdanau et al. introduced a technique that allowed the decoder to focus on the appropriate words (as encoded by the encoder) at each time step

Attention Mechanisms

- So the path from an input word to its translation
  - Is now much shorter
  - So short-term memory limitations of RNNs have much less impact

CLOUD x LAB

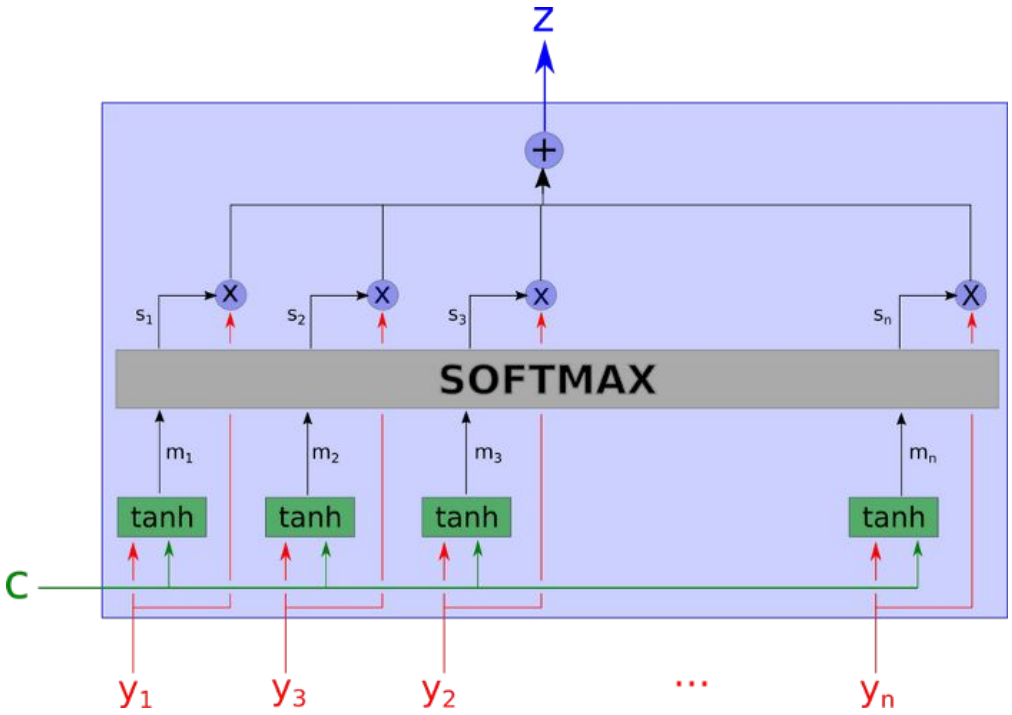
Attention Mechanisms



Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

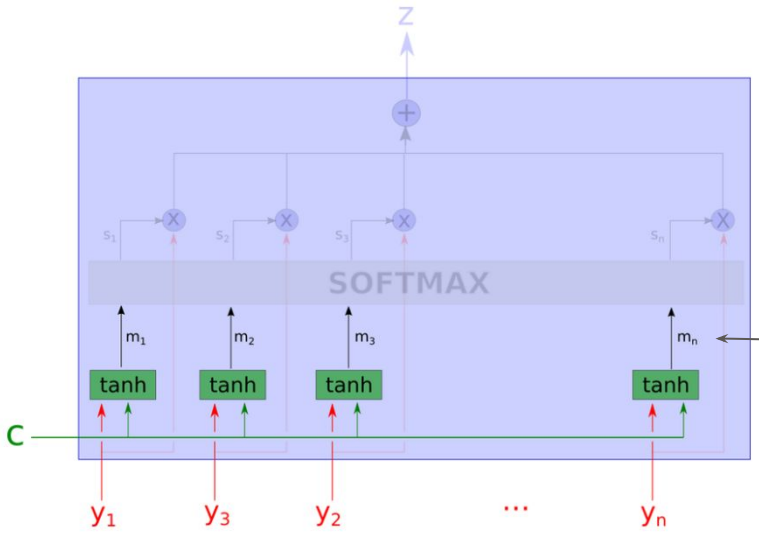
CLOUD x LAB

Attention Mechanisms



Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

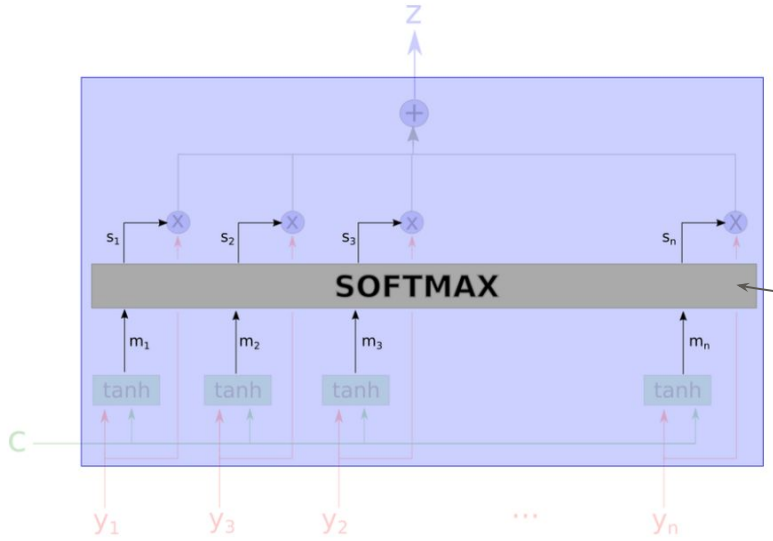
Attention Mechanisms



$$m_i = \tanh(W_{cm}c + W_{ym}y_i)$$

Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

Attention Mechanisms



When 1 output saying which input is max, like below –

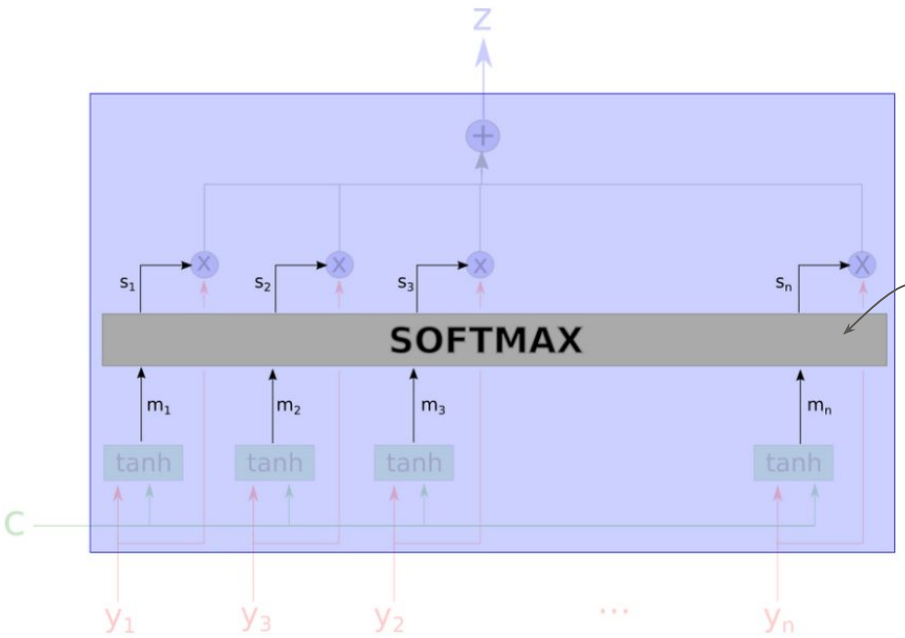
$$\text{argmax}(x_1,\ldots,x_n) = (0,\ldots,0,1,0,\ldots,0)$$

Then –

$$\text{softmax}(x_1,\ldots,x_n) = (e^{x_i}/\sum e^{x_j})_i$$

Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

Attention Mechanisms



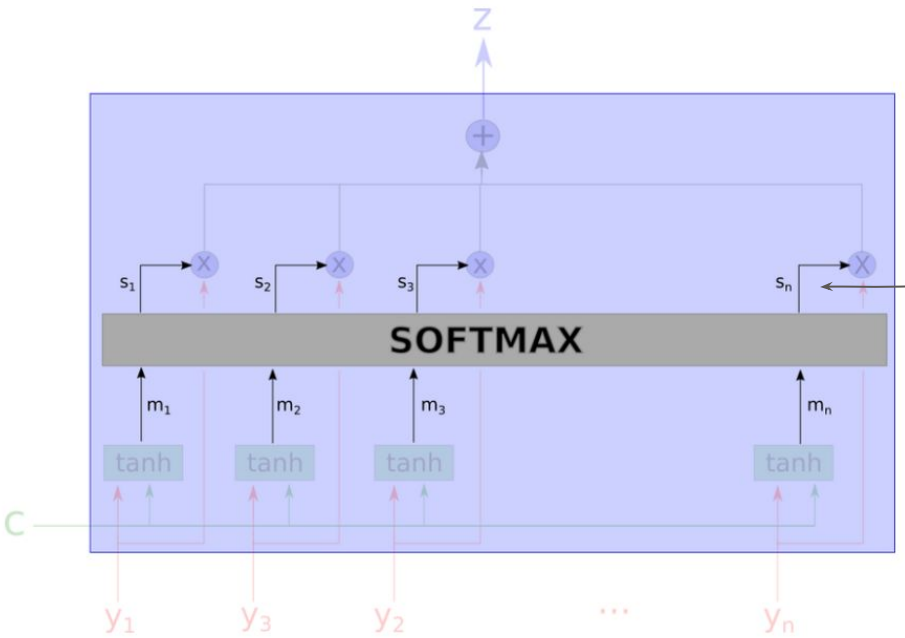
When one x_i is bigger than other –

Then –

$\text{softmax}(x_1,\ldots,x_n) \cong \text{argmax}(x_1,\ldots,x_n)$

Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

Attention Mechanisms



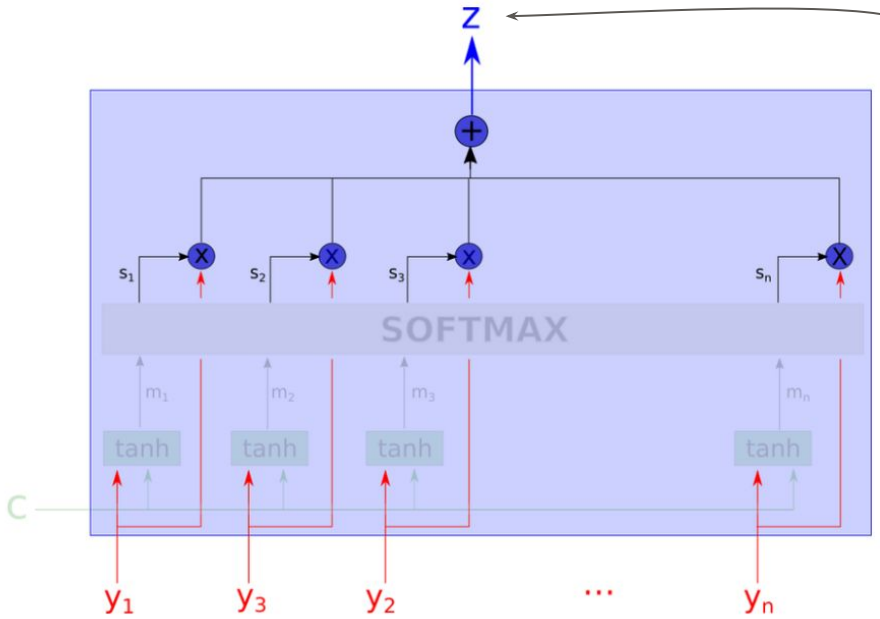
$s_i \propto \exp(\langle w_m, m_i \rangle)$

$\sum_i s_i = 1$

s_i are the softmax of the m_i projected on a learned direction

Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

Attention Mechanisms



$z = \sum_i s_i y_i$

z is the weighted arithmetic mean of all the y_i

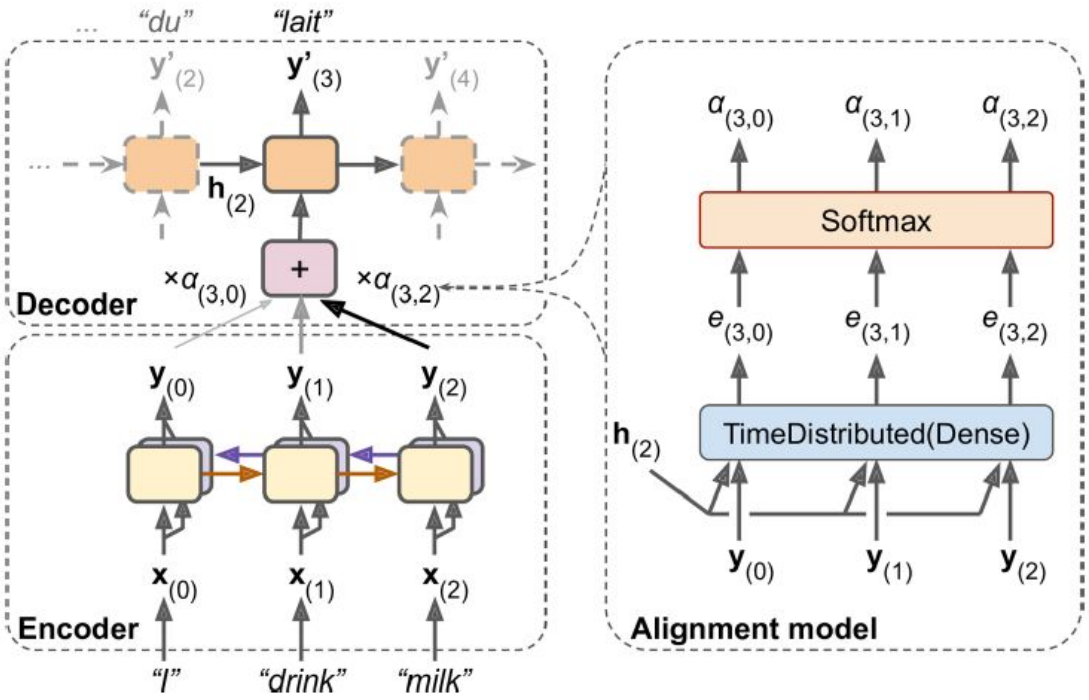
Source: https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

Attention Mechanisms - References

- https://arxiv.org/abs/1604.03736
- https://distill.pub/2016/augmented-rnns/
- http://akosiorek.github.io/ml/2017/10/14/visual-attention.html
- https://medium.com/heuritech/attention-mechanism-5aba9a2d4727

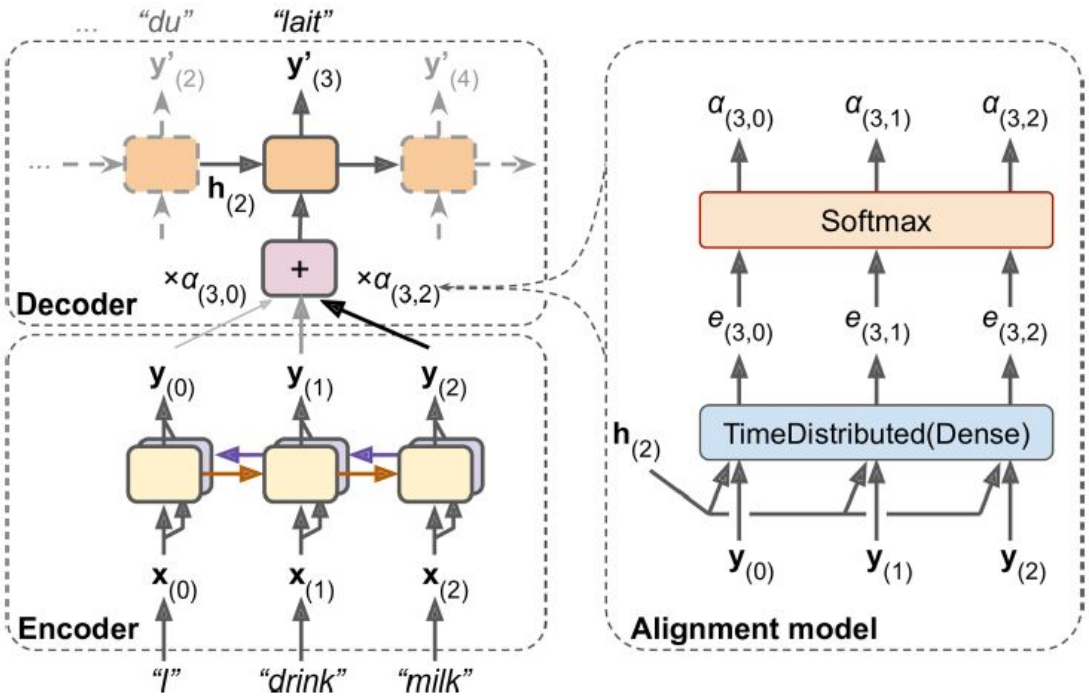
CLOUD x LAB

Attention Mechanisms



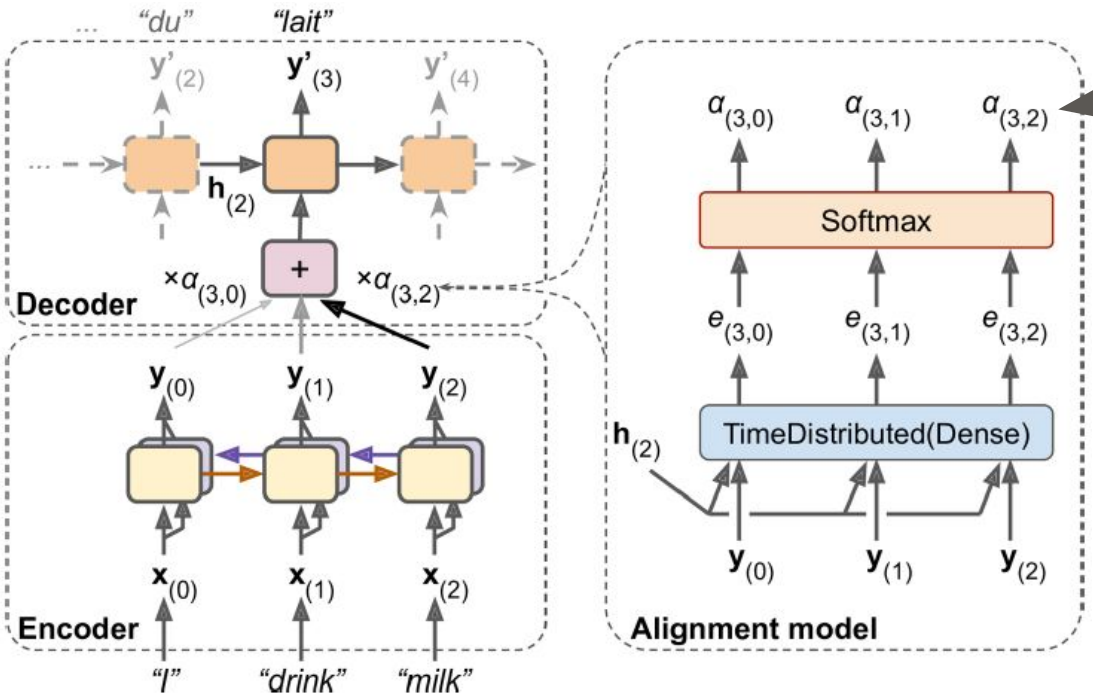
This shows this model's slightly simplified architecture

Attention Mechanisms



Instead of just sending the encoder's final hidden state to the decoder, we now send all of its outputs to the decoder

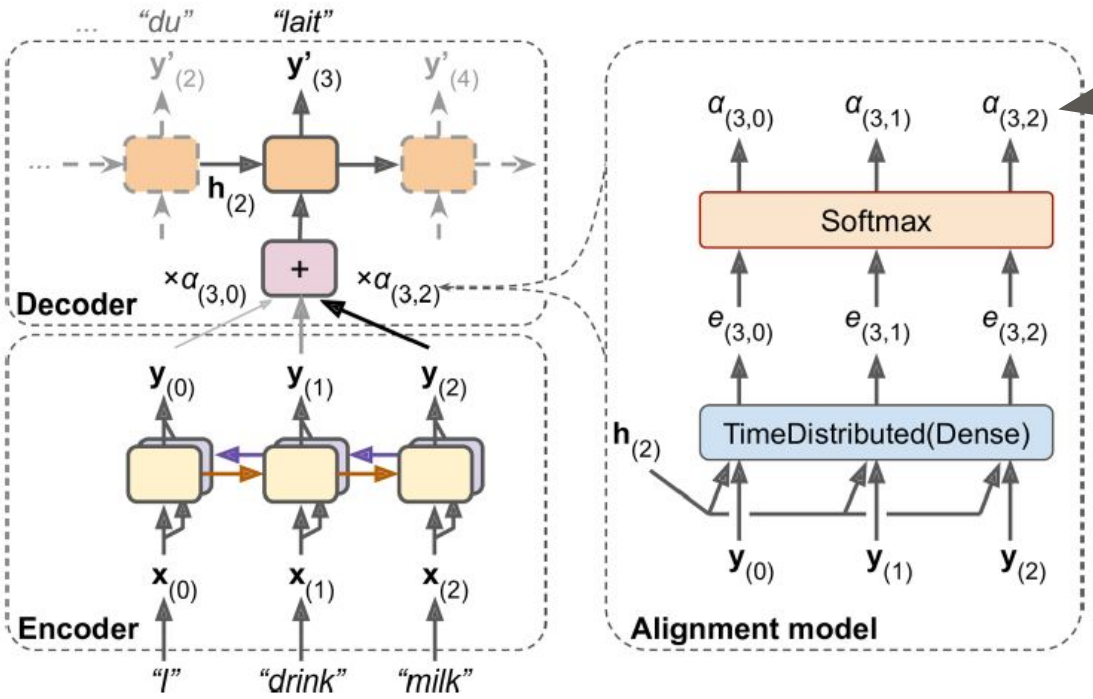
Attention Mechanisms



If the weight α(3,2) is much larger than the weights α(3,0) and α(3,1), then the decoder will pay much more attention to word number 2 ("milk") than to the other two words

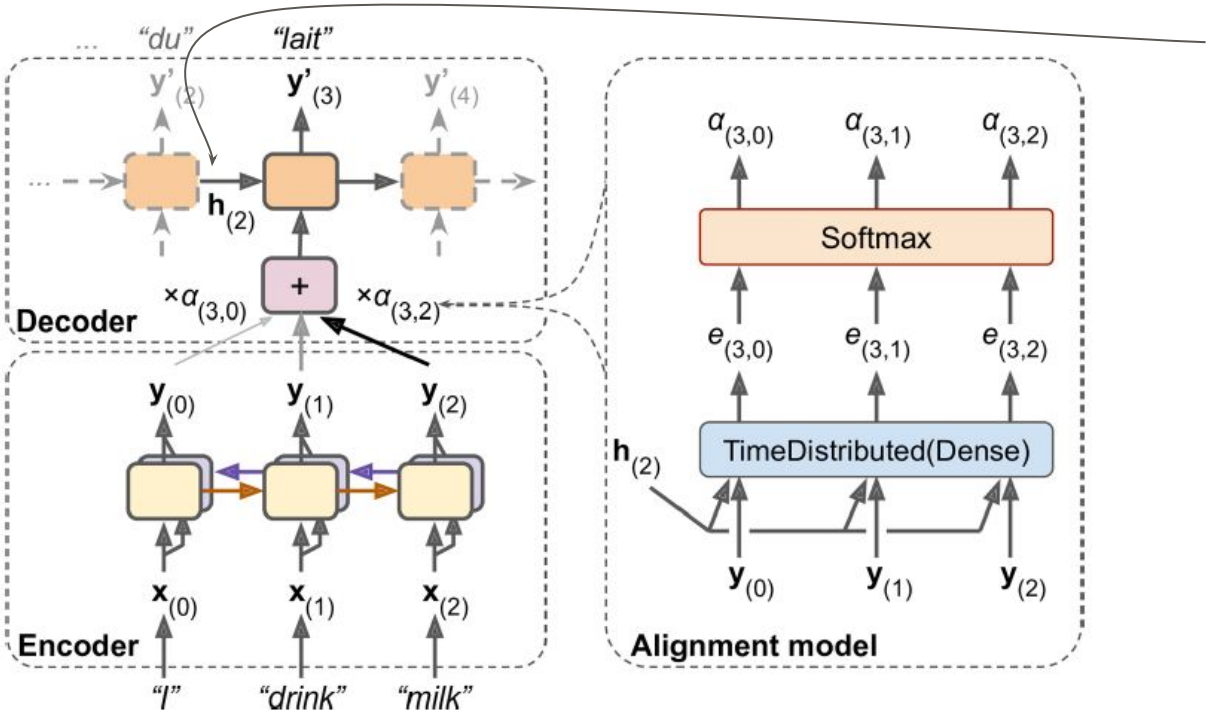
CLOUD x LAB

Attention Mechanisms



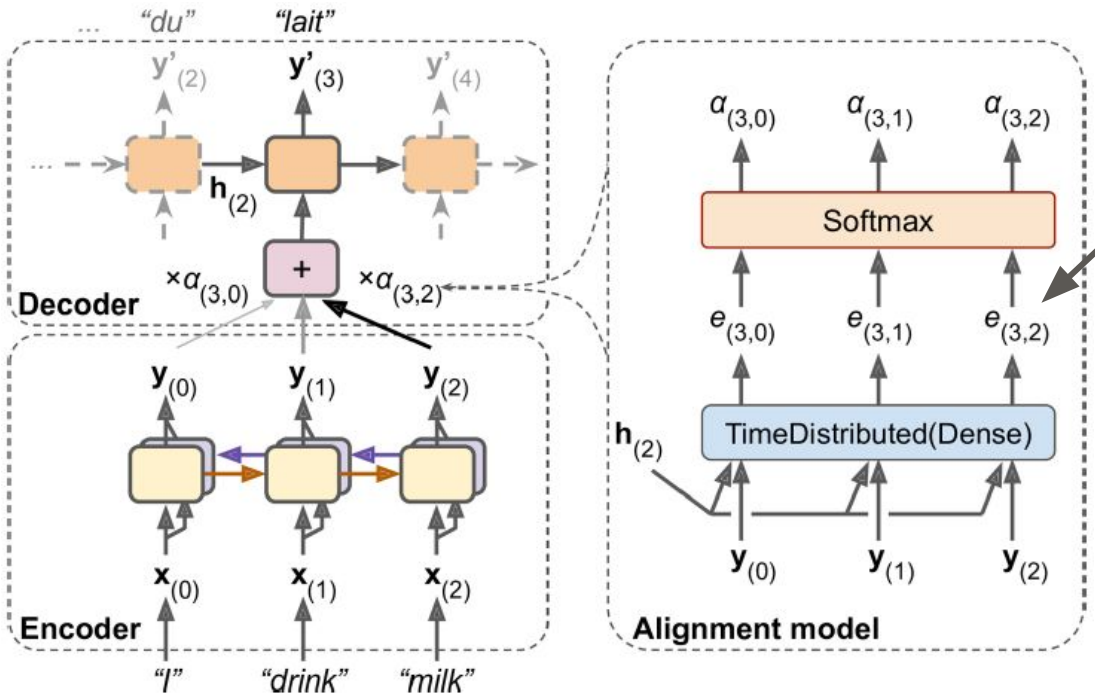
These weights are generated by a type of small neural network called an alignment model (or an **attention layer**), which is trained jointly with the rest of the Encoder–Decoder model

Attention Mechanisms



It starts with a time-distributed `Dense` layer with a single neuron, which receives as input all the encoder outputs, concatenated with the decoder's previous hidden state $h_{(2)}$

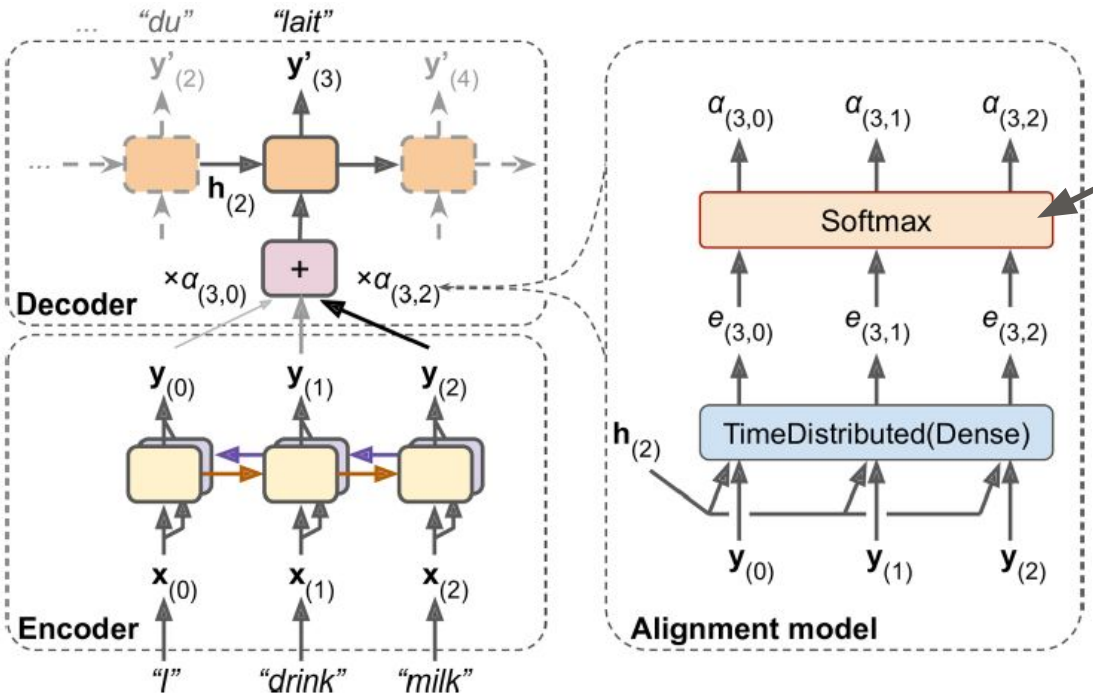
Attention Mechanisms



This layer outputs a score (or energy) for each encoder output $e_{(3, 2)}$

This score measures how well each output is aligned with the decoder's previous hidden state

Attention Mechanisms



Finally, all scores go through softmax layer to get final weight for each encoder output $\alpha_{(3,2)}$

All weights for a given decoder time step add up to 1 (since the softmax layer is not time-distributed)

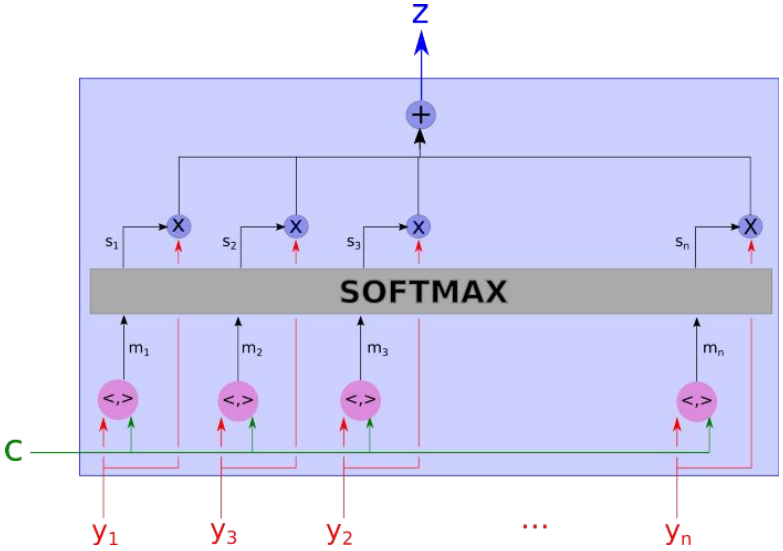
Attention Mechanisms

- This attention mechanism is called **Bahdanau attention** which
- Concatenates encoder's output with decoder's previous hidden state
- So it is called **concatenative attention** (or additive attention)

Attention Mechanisms

- Another common attention mechanism proposed in 2015 by Minh-Thang Luong et al.
- It's called **Luong attention** or **multiplicative attention**.

m = c.y



CLOUD x LAB

Attention Mechanisms

- Here is how you can add Luong attention to an Encoder–Decoder model using TensorFlow Addons:

```
attention_mechanism = tfa.seq2seq.attention_wrapper.LuongAttention(
    units, encoder_state, memory_sequence_length=encoder_sequence_length)
attention_decoder_cell = tfa.seq2seq.attention_wrapper.AttentionWrapper(
    decoder_cell, attention_mechanism, attention_layer_size=n_units)
```

Note: This code is given just for explanation and is not implemented in the notebook.

CLOUD x LAB

Attention Mechanisms

Benefits of attention mechanisms:

- Significant improvement for long sentences
- Make it easier to understand what led the model to produce its output
  - This is called **explainability**

Visual Attention

Visual Attention

- Attention mechanisms are now used for a variety of purposes
  - For example, generating captions by looking at images

Visual Attention

- A model generated the caption “A woman is throwing a frisbee in a park” for the below image:

CLOUD x LAB

Visual Attention

- CNN processes the image
- Outputs feature maps
- Then decoder RNN equipped with attention mechanism
    - Generates the caption, one word at a time

At each decoder time step (each word), the decoder uses the attention model to focus on just the right part of the image

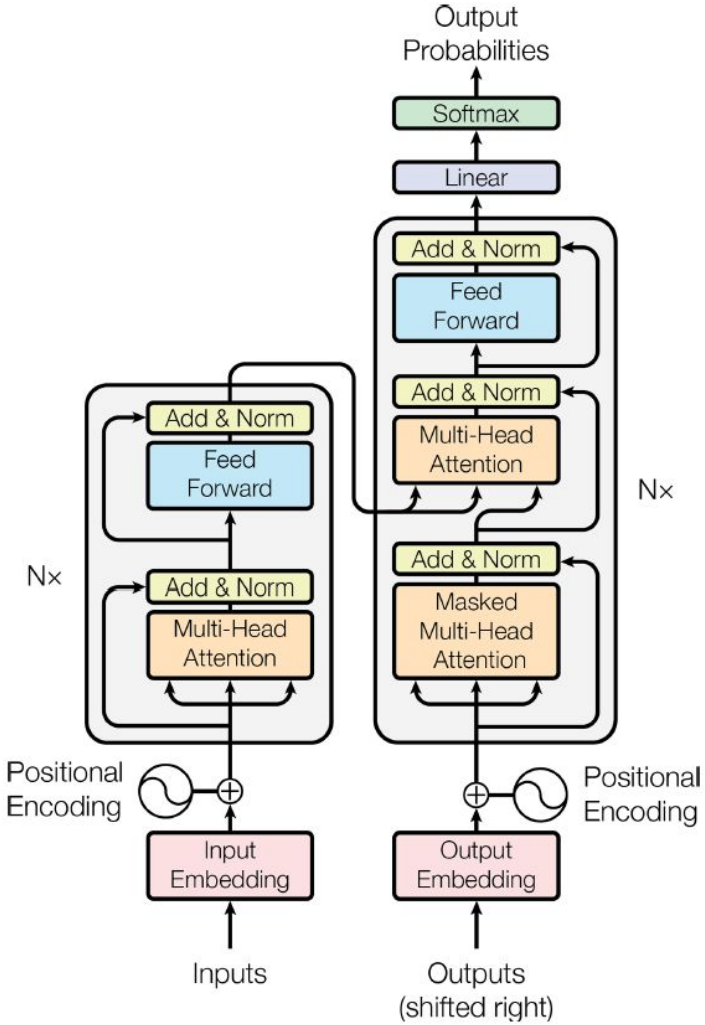
CLOUD x LAB

The Transformer Architecture

The Transformer Architecture

- In 2017, a team of Google researchers created an architecture called the **Transformer**
- It used only Attention Mechanism, no RNN or CNN to accompany it

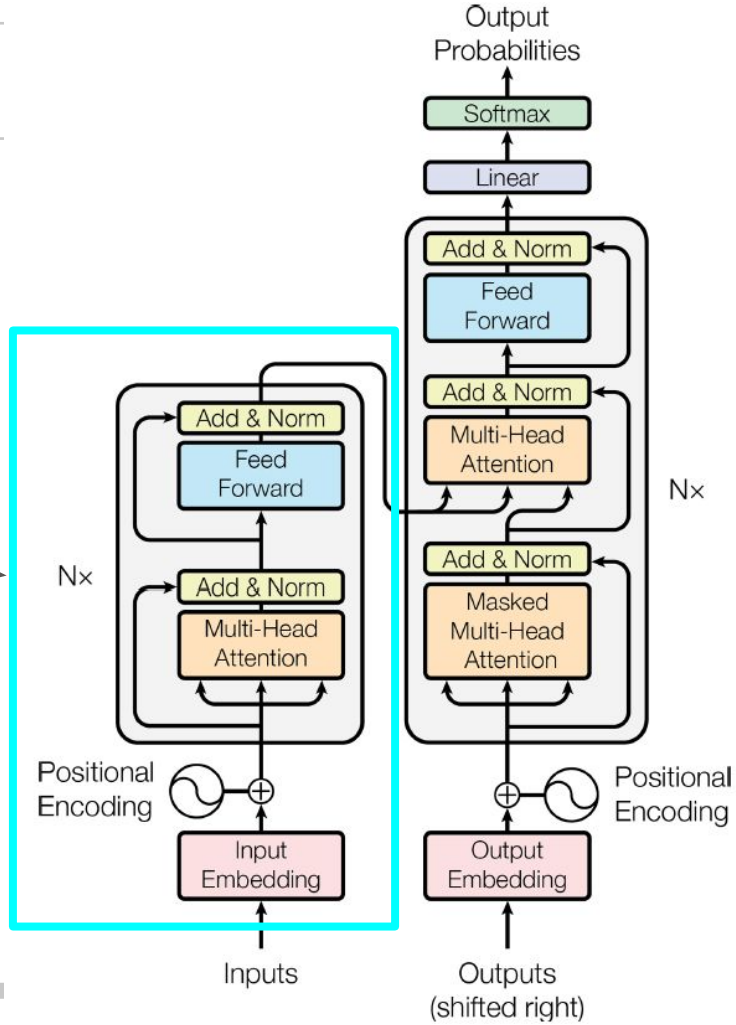
The Transformer Architecture



CLOUD x LAB

The Transformer Architecture

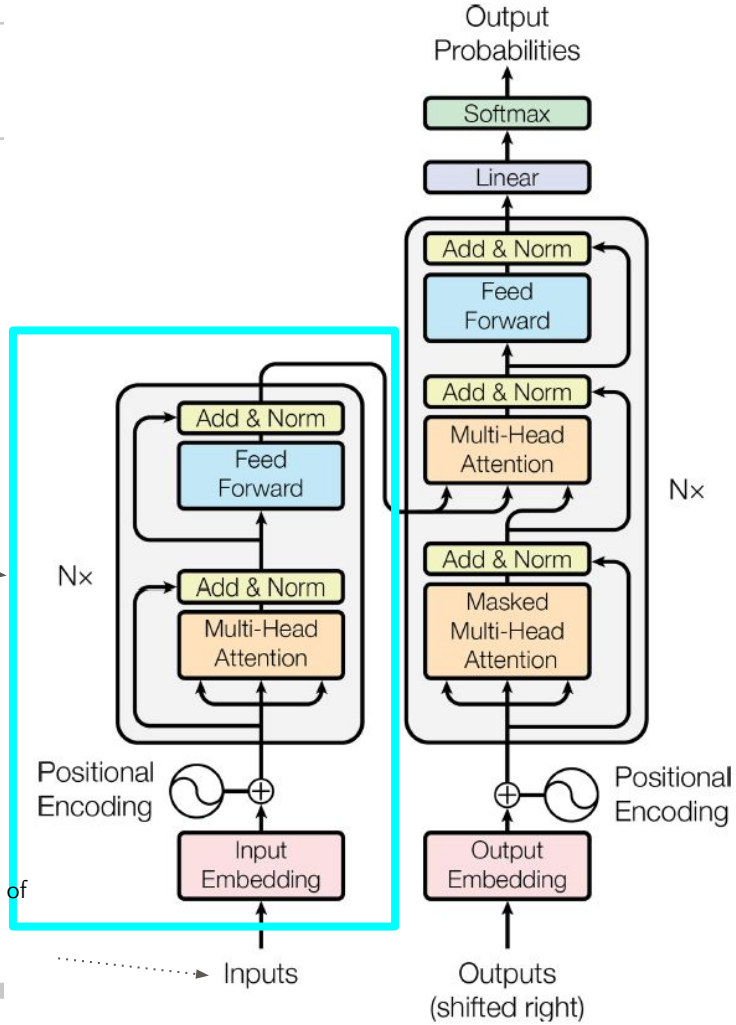
Encoder



CLOUD x LAB

The Transformer Architecture

Encoder

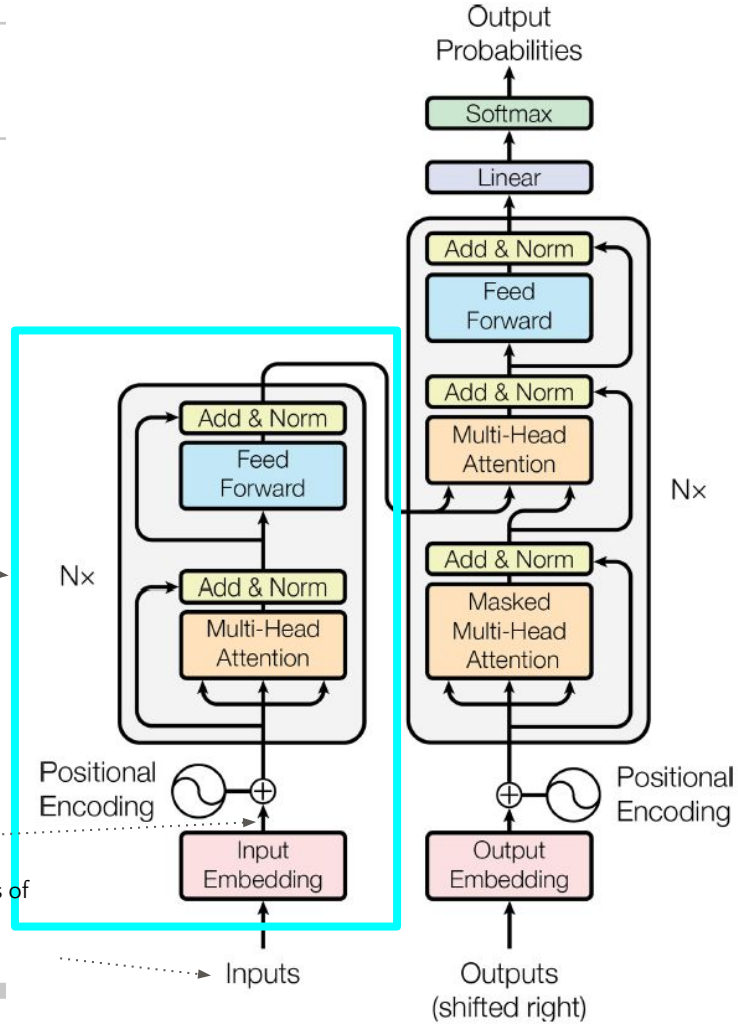


A batch of sentences represented as sequences of word IDs.
Shape: **[batch size, max input sentence length]**

CLOUD x LAB

The Transformer Architecture

Encoder



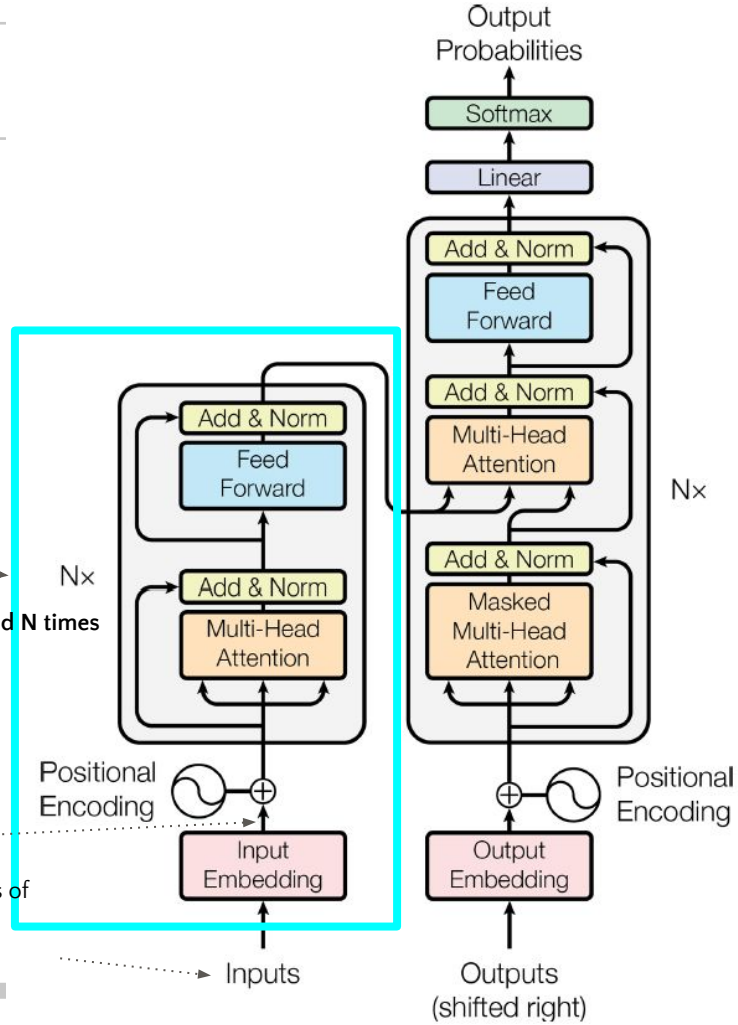
[batch size, max input sentence length, 512]

A batch of sentences represented as sequences of word IDs.
Shape: **[batch size, max input sentence length]**

CLOUD x LAB

The Transformer Architecture

Encoder



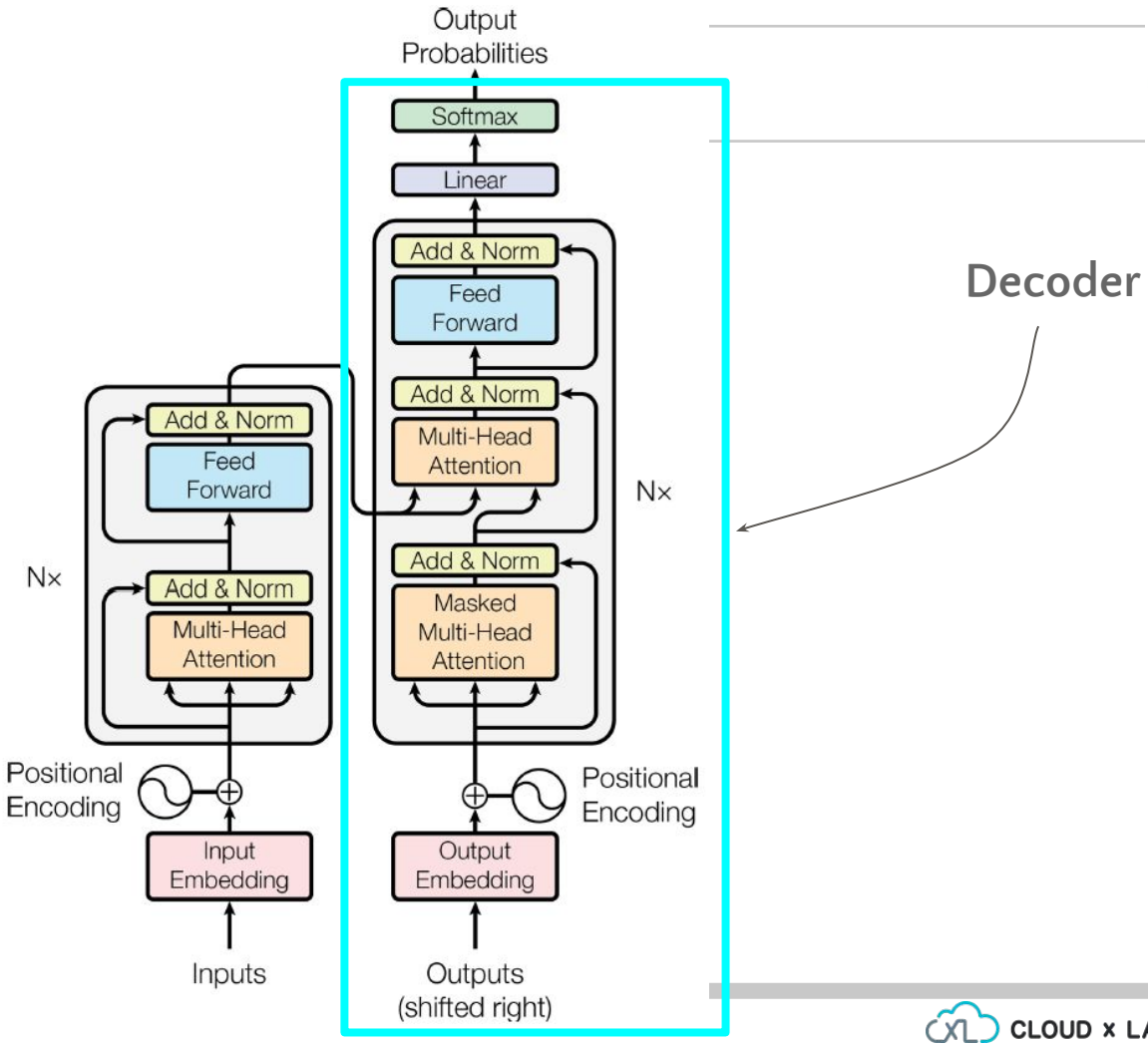
Stacked N times
N = 6

[batch size, max input sentence length, 512]

A batch of sentences represented as sequences of word IDs.
Shape: **[batch size, max input sentence length]**

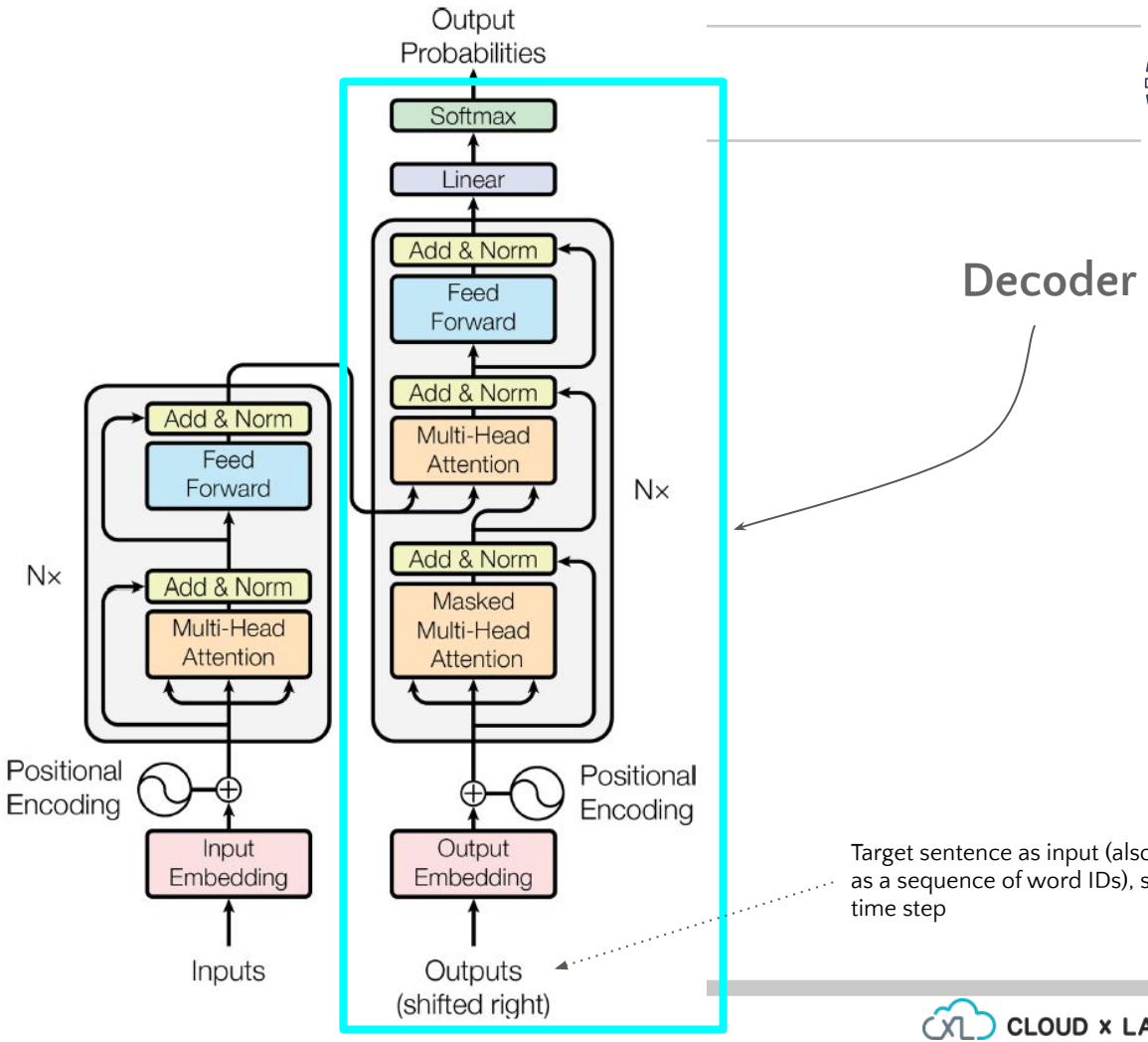
CLOUD x LAB

The Transformer Architecture



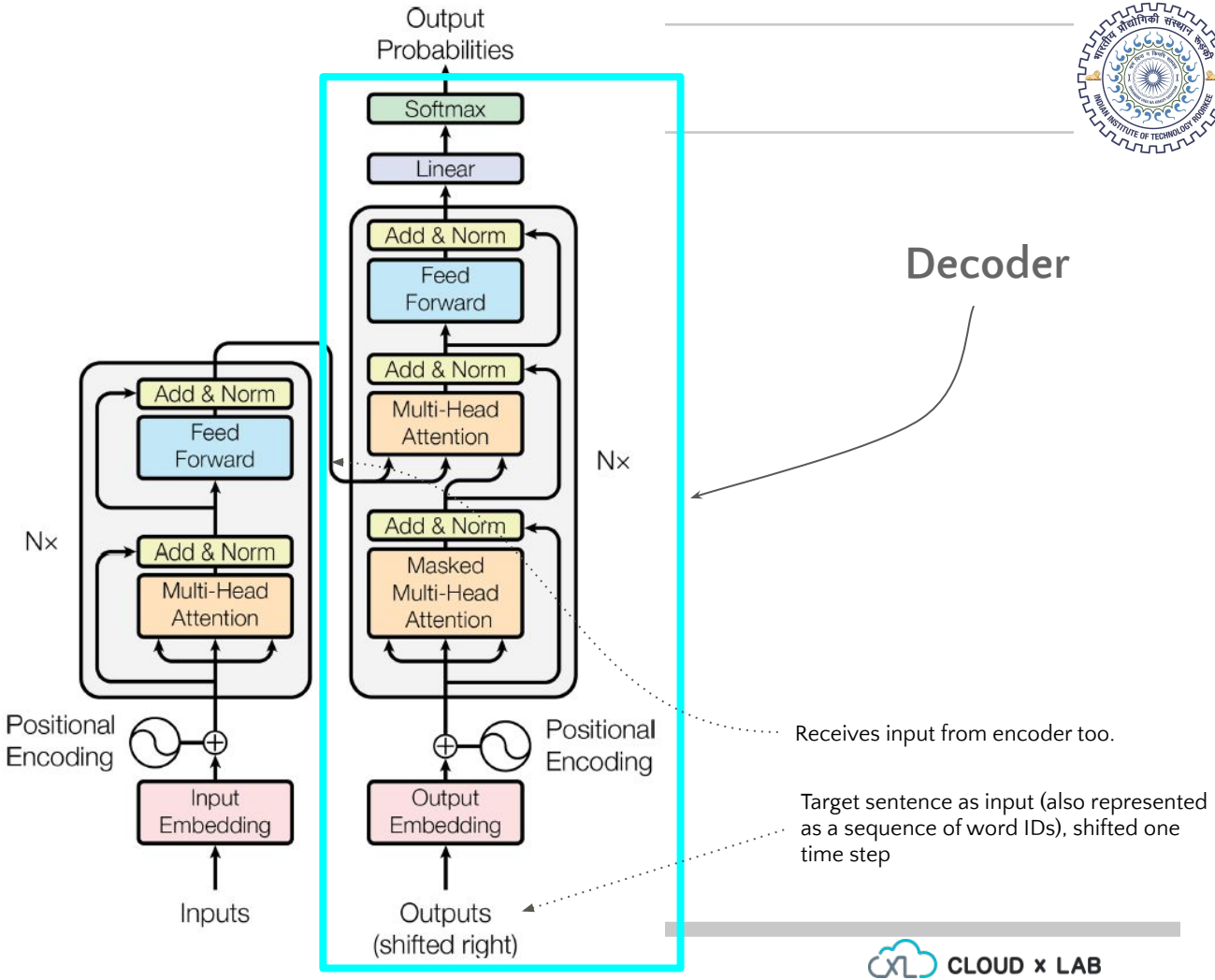
CLOUD x LAB

The Transformer Architecture



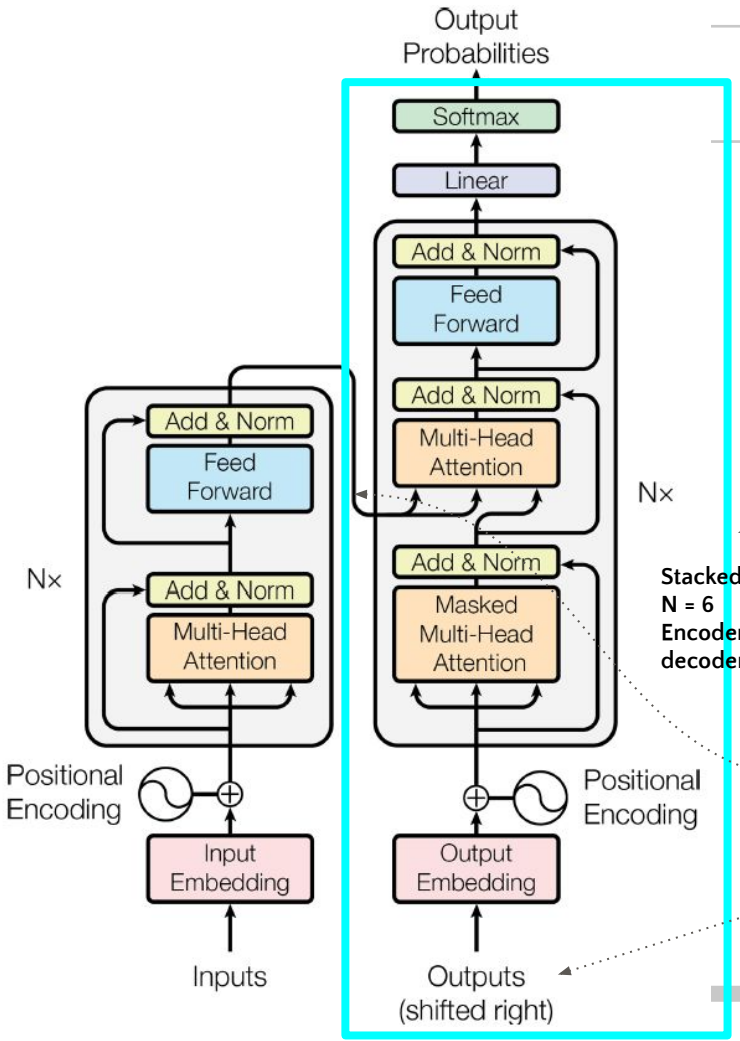
CLOUD x LAB

The Transformer Architecture



CLOUD x LAB

The Transformer Architecture



Decoder

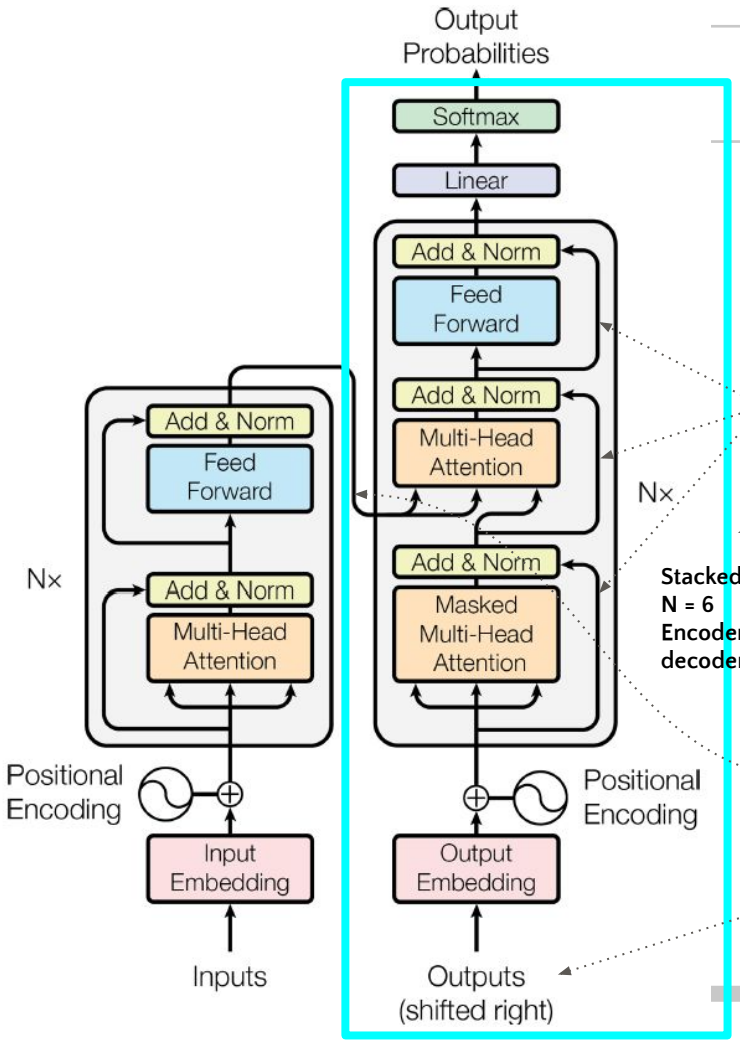
Stacked N times
N = 6
Encoder stack's final outputs are fed to the decoder at each of these N levels.

Receives input from encoder too.

Target sentence as input (also represented as a sequence of word IDs), shifted one time step

CLOUD x LAB

The Transformer Architecture



Decoder

Skip Conn.

Stacked N times
N = 6
Encoder stack's final outputs are fed to the decoder at each of these N levels.

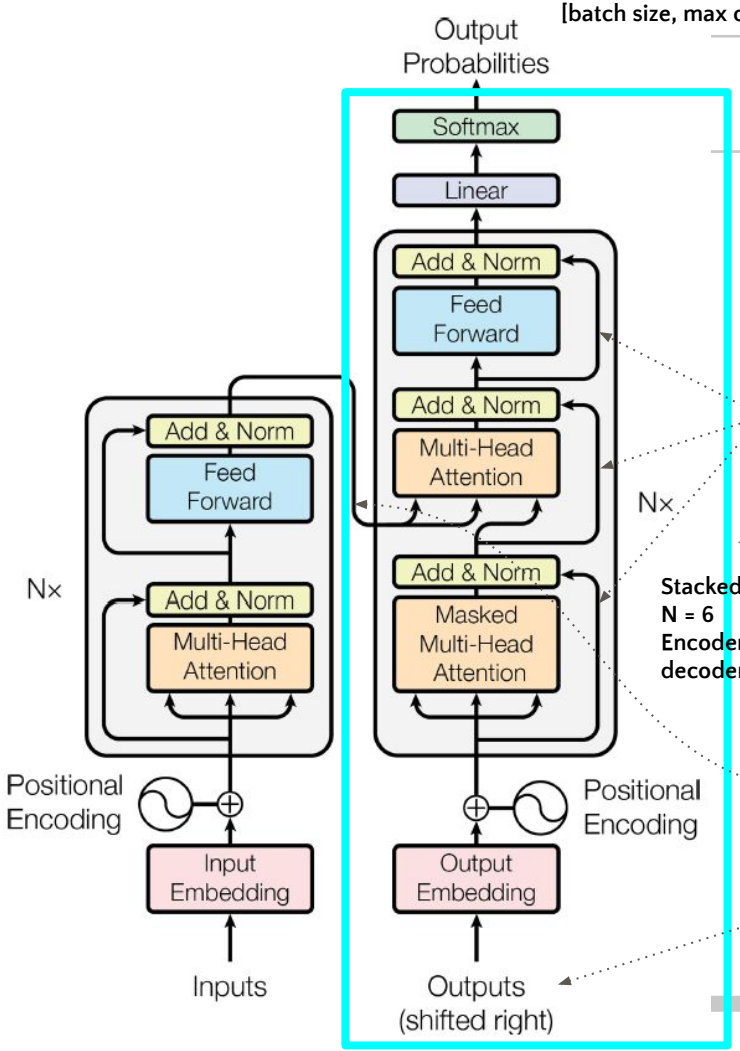
Receives input from encoder too.

Target sentence as input (also represented as a sequence of word IDs), shifted one time step

CLOUD x LAB

[batch size, max output sentence length, vocabulary length]

The Transformer Architecture



Decoder

Skip Conn.

Stacked N times
N = 6
Encoder stack's final outputs are fed to the decoder at each of these N levels.

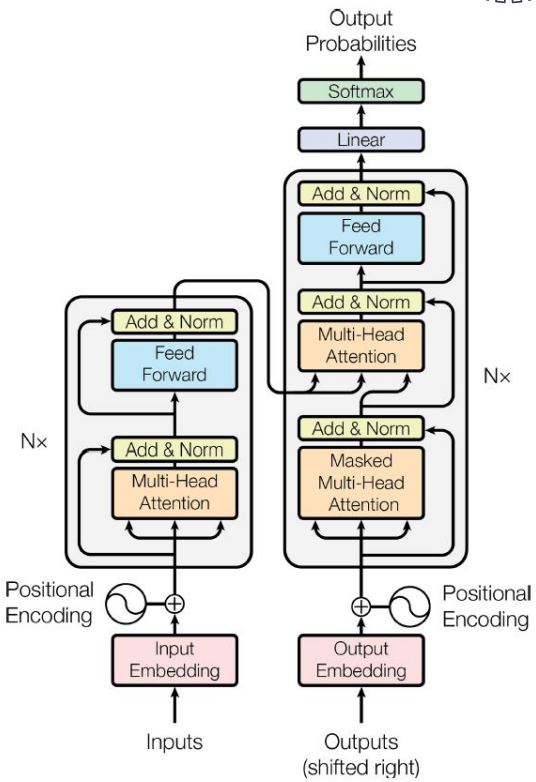
Receives input from encoder too.

Target sentence as input (also represented as a sequence of word IDs), shifted one time step

CLOUD x LAB

The Transformer Architecture (More)

- Two embedding layers
- 5 × N skip connections
- Each of them followed by a layer normalization layer
- 2 × N "Feed Forward" – 2 dense layers each
  - First one using the ReLU
  - second with no activation function
- The output layer is a dense layer using the softmax
- All of these layers are time-distributed.
  - So, each word is treated independently



The Transformer Architecture

- We are familiar with most components, except 2 of them:
  - Multi-Head Attention layer
  - Positional embeddings

Positional embeddings

References:

1. https://arxiv.org/pdf/1706.03762.pdf
2. https://kazemnejad.com/blog/transformer_architecture_positional_encoding/
3. https://www.tensorflow.org/tutorials/text/nmt_with_attention

Positional embeddings

Consider the 2 following sentences:

> I **do not** like the story of the movie, but I **do** like the cast

> I **do** like the story of the movie, but I **do not** like the cast

What is the difference between these 2 sentences?

Souce: https://is.gd/positionalembedding

Positional embeddings

- The words are same
- Meaning is different
- Information of order is required to distinguish different meanings

Souce: https://is.gd/positionalembedding

Positional embeddings

- **Positional embedding** generates embeddings
- Using sin and cos functions
- Which allows the model to learn the relative positions of words

Souce: https://is.gd/positionalembedding

Positional embeddings

- Positional embedding matrix P is defined as:

$$P_{p,2i} = \sin\left(p/10000^{2i/d}\right)$$
$$P_{p,2i+1} = \cos\left(p/10000^{2i/d}\right)$$

Here $P_{p,i}$ is the *ith* component of the embedding for the word located at the *pth* position in the sentence.

CLOUD x LAB

Positional embeddings

There is no `PositionalEmbedding` layer in TensorFlow,

but it is easy to create one.

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_ii.ipynb

CLOUD x LAB

Multi-Head Attention

Multi-Head Attention

- **Multi-head attention** allows the model to
- Jointly attend to information
    - from different representation subspaces
    - at different positions
- With a single attention head, averaging inhibits this.

Multi-Head Attention

- Encodes each word's relationship with every other in a sentence
- Paying more attention to the most relevant ones
- Called **Self-Attention**

For example:

- "They welcomed the Queen of the United Kingdom"
- The output for "Queen" will depend on all the words in the sentence,
- but it will probably pay more attention to
    - "United" and "Kingdom" than
    - "They" or "welcomed."

CLOUD x LAB

Multi-Head Attention

- Before we get into Multi-Headed Attention, we should first look at the concept of **Scaled Dot-Product Attention**
- **Example:**
-

Multi-Head Attention - Scaled Dot-Product Attention

- Say, encoder analyzed “They played chess,” and understood:
  - “They” → Subject
  - "Played" → verb
- This is encoded in the representations of the words
- Say, decoder has already translated the subject
  - and it thinks that it should translate the verb next
  - For this, it needs to fetch the verb from the input sentence.
- This is similar to a dictionary lookup:
  - Look up “verb” in {“subject”: “They”, “verb”: “played”, ...}

CLOUD x LAB

Multi-Head Attention - Scaled Dot-Product Attention

- However, the model has
  - **vectorized representations** of these concepts
  - Instead of discrete tokens
    - to represent the keys (like “subject” or “verb”)
- Solution:
  - Lookup <> similarity measure between the query and each key in the dictionary, and
  - Then use the softmax function to convert these similarity scores to weights that add up to 1.
  - The result ~ 1 means the query has given result
- **In short, you can think of this whole process as a differentiable dictionary lookup.**

Multi-Head Attention

- Compared to the standard form of Attention, Scaled Dot-Product Attention utilizes Scaled Dot-Product to calculate similarity

$$\text{Attention}\,(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

Multi-Head Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

- Q is a matrix containing one row per query of shape $[n_{queries}, d_{keys}]$
- $n_{queries}$ is the number of queries
- d_{keys} is the number of dimensions of each query and each key

CLOUD x LAB

Multi-Head Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

- K is a matrix containing one row per key of shape $[n_{keys}, d_{keys}]$
- n_{keys} is the number of keys and values

CLOUD x LAB

Multi-Head Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

- V is a matrix containing one row per value of shape $[n_{keys}, d_{values}]$
- d_{values} is the number of each value

Multi-Head Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

- The shape of QK^T is $[n_{queries}, n_{keys}]$
- It contains one similarity score for each query/key pair

CLOUD x LAB

Multi-Head Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_{keys}}}\right)\mathbf{V}$$

- The output of the softmax function has the same shape, but all rows sum up to 1
- The final output has a shape of $[n_{queries}, d_{values}]$, there is one row per query, where each row represents the query result (a weighted sum of the values)

CLOUD x LAB

Multi-Head Attention

- The `keras.layers.Attention` layer implements Scaled Dot-Product Attention
- Its inputs are just like Q, K, and V, except with an extra batch dimension (the first dimension)

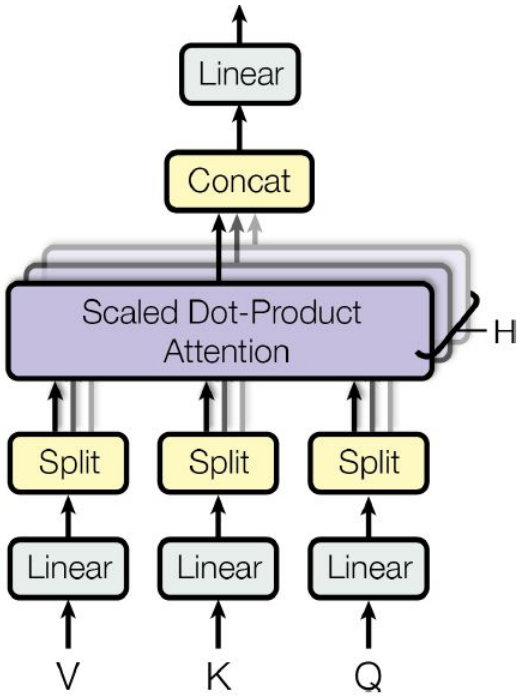
Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_ii.ipynb

CLOUD x LAB

Multi-Head Attention

- Now let's look at the Multi-Head Attention layer's architecture



V = Value

K = Key

Q = Query

CLOUD x LAB

Multi-Head Attention

- It is a bunch of Scaled Dot-Product Attention layers
- Each preceded by linear transformation of the values, keys, and queries
- All outputs are concatenated
- And they go through a final linear transformation

Multi-Head Attention

But why?

What is the intuition behind this architecture?

Multi-Head Attention

- The word representation encodes many different characteristics of the word
- With a single Scaled Dot-Product Attention layer this is not possible

Multi-Head Attention

- This is why Multi-Head Attention layer applies multiple different linear transformations of values, keys, and queries
- It gives the attention layer multiple “representation subspaces”

Multi-Head Attention

How to implement a multi-head attention in Keras?

Switch to Notebook

nlp_with_rnns_and_attention/nlp_with_rnns_and_attention_ii.ipynb

CLOUD x LAB

Recent Innovations in Language Models

Recent Innovations in Language Models

- The year 2018 has been called the “ImageNet moment for NLP”: progress was astounding, with larger and larger LSTM and Transformer-based architectures trained on immense datasets

Recent Innovations in Language Models

You check out the following papers, all published in 2018:

- The ELMo paper by Matthew Peters introduced Embeddings from Language Models (ELMo)

- The ULMFiT paper by Jeremy Howard and Sebastian Ruder demonstrated the effectiveness of unsupervised pretraining for NLP tasks

Recent Innovations in Language Models

You check out the following papers, all published in 2018:

- The GPT paper by Alec Radford and other OpenAI researchers also demonstrated the effectiveness of unsupervised pretraining using a Transformer-like architecture. GPT - 3 released on June 11, 2020.

- The BERT paper by Jacob Devlin and other Google researchers also demonstrates the effectiveness of self-supervised pretraining on a large corpus, using a similar architecture to GPT but non-masked Multi-Head Attention layers.

Questions?

https://discuss.cloudxlab.com

reachus@cloudxlab.com